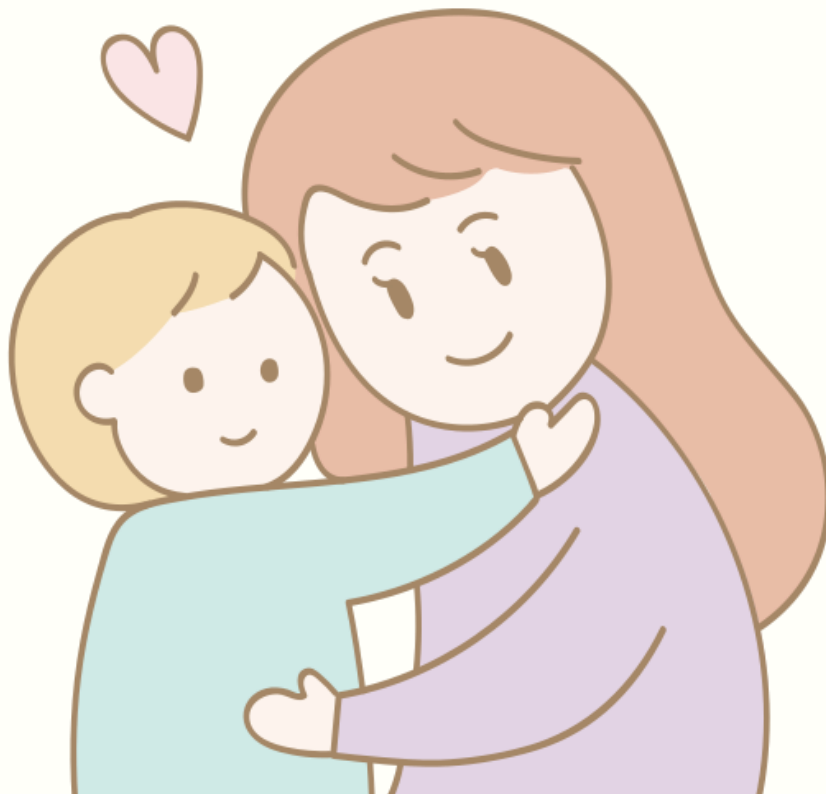


“흩어진 육아서비스를 하나의 것으로”

아이봄

Team 기적의세대

“AI가 함께하는 올인원 육아 케어 플랫폼”



팀장 | 황용현

팀원 | 권용익, 이민주, 이재원, 임대한, 윤승진

Github | github.com/sai0734/Team_The-Worst-Generation

시연 영상 |

<https://www.youtube.com/watch?v=pjrz7tYPWqI>

프로젝트 기간 | 2026.08.03 ~ 2026.08.30

Context

- ① 프로젝트 개요
- ② 시스템 설계
- ③ 핵심 기능 구현
- ④ 시스템 최적화 및 성능 개선
- ⑤ 문제 해결 및 회고



프로젝트개요

“늘어나는 출산율, 부모의 부담은 줄이고 편의성은 늘린다”

기획 배경

- 높아지는 출산율, 커지는 육아 정보 수요
- 늘어나는 육아 플랫폼, 여전히 흩어진 기능
- 활용되지 못하는 육아 기록



프로젝트 목표

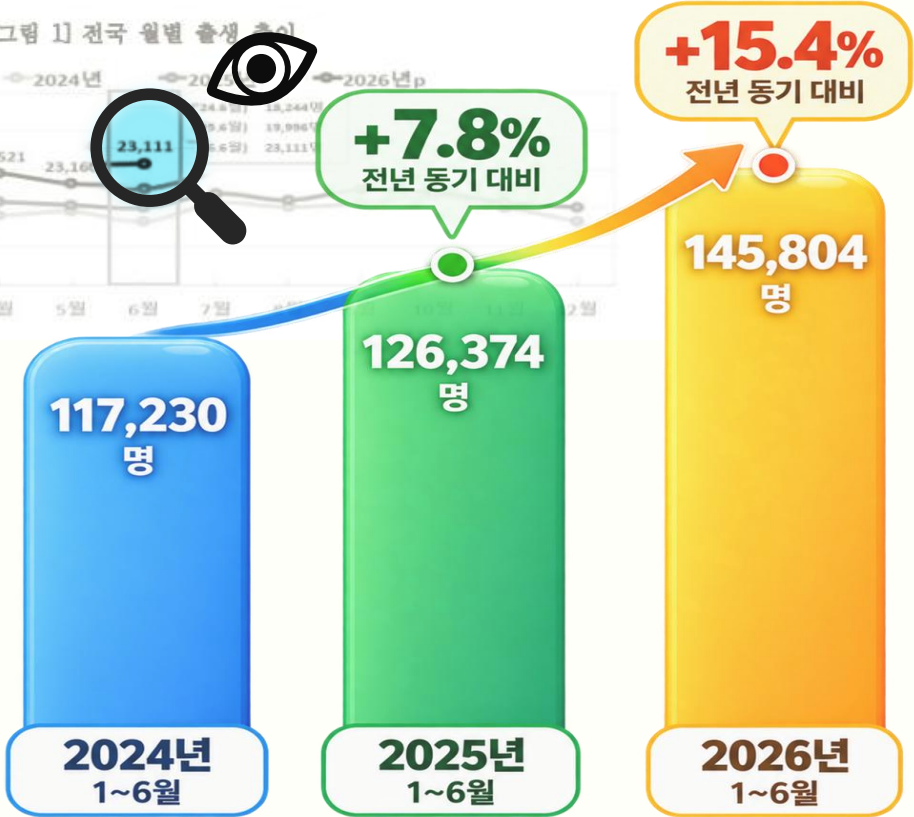
- 흩어진 육아 서비스를 하나의 플랫폼에 통합하고,
- AI 기능으로 관리 부담은 줄이고 편의성은 더하게

□ 2026년 6월 출생아 수는 23,111명, 전년동월대비 3,115명(15.6%) 증가함

[표 1] 전국 출생아 수 및 증감률

	2024년	2025년	2025년		2026년p			
			6월	1~6월	4월	5월	6월	1~6월
출생아 수(명)	238,317	254,341	19,996	126,374	24,521	23,160	23,111	145,804
전년(동월) 증감률(%)	3.6	6.7	9.6	7.8	18.1	13.8	15.6	15.4

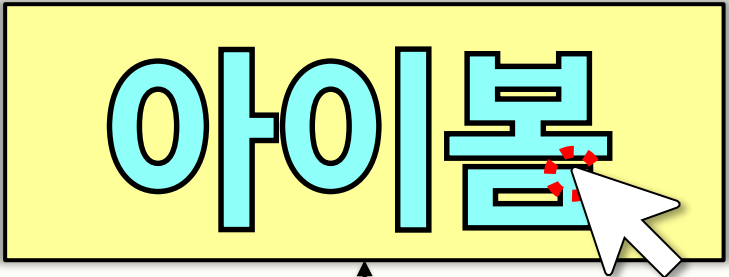
[그림 1] 전국 월별 출생아 수



차별점 및 주요 서비스 흐름

“여러 앱을 오가지 않고, 하나의 플랫폼에서 육아 전 과정을 관리”

차별점? “올인원 플랫폼”



서비스를 한 흐름으로 ✓

기록 (Record)



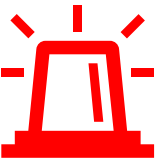
일기·가계부·건강기록등록

AI 분석 (AI Insight)

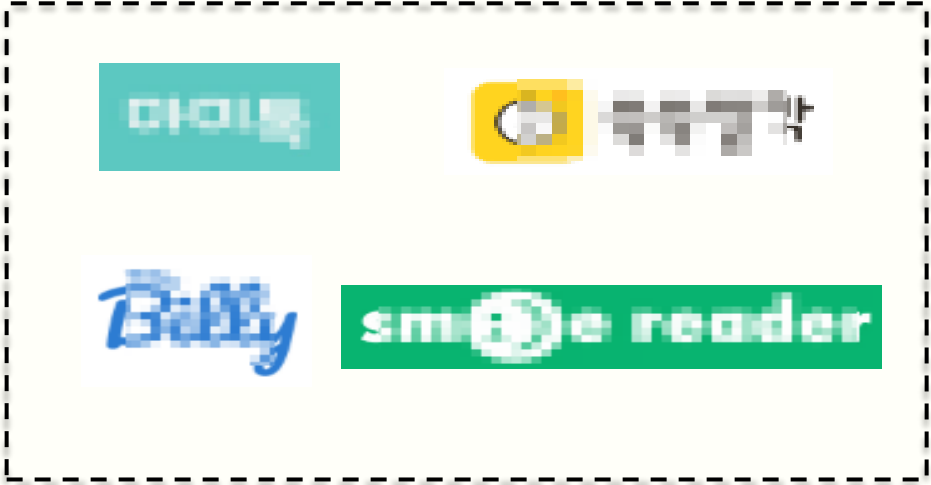


ChatBot·사진분석
·리콜매칭·위험감지

케어·알림 (Care)



SMS·리포트·커뮤니티공유



구분	기존 육아서비스	아이봄
서비스형태	기능별로여러앱/사이트분산	하나의플랫폼에통합제공
기록관리	각앱에기록이흩어짐	일기·가계부등한곳에서기록
리콜확인	직접찾아보고수동확인	등록제품자동매칭+SMS
육아상담	커뮤니티검색·직접문의	AI챗봇실시간상담
사진분석	보호자수작업판단	AI가피부·대변상태자동분석
안전관리	보호자가직접확인	홈캠사이탈감지자동알림

팀구성및역할

이름	역할	Python/OnDevice	Backend	Front-end
황용현	팀장	AI행동교정-Ollamaqwen3 수면패턴분석-sdkitlearn+qwen3 육아일기사진분석-VARCOVISION 동영상생성-moviepy+facebook/immstsakor	아이등록·성장·접종·수면-CRUD,이미지파일처리 일기·앨범·인화주문-CRUD,페이징·Toss결제(API) AI행동교정-Naver·YouTubeAPI+qwen3결과처리	아이등록·대시보드·성장·접종·수면시각화(Recharts) 일기·앨범·무한스크롤·페이징,Toss결제(SDK) 이미지정보추출(exif),이미지파일드래그앤드롭 AI행동교정-뉴스/영상+3단계표시및지난기록 커스텀아이정보상태공용화(RTK)
권용익	팀원	SMS자동알림연동-OpenClaw 리콜유사도매칭-Sentence_Transformers 가계부자동분류·브리핑-SpringAI(ChatClient) 커뮤니티게시글AI요약-Ollama	리콜매칭·SMS발송·인증·국내·해외리콜정보 대조,SMS발송 가계부-CRUD 베이비시터-CRUD·요청·WebSocket채팅 커뮤니티게시판·댓글·좋아요CRUD	리콜제품·등록·리스트·상세화면 가계부-AI자동분류·브리핑 베이비시터·거리기반검색·채팅 구안글·다중요일선택 커뮤니티게시판·댓글
이민주	팀원	성분표검사-GoogleVisionOCR 건강체크-Ollamaqwen2.5vl7b 산책로추천-Ollamaparentingqwen8b	알레르기성분표검사·이미지OCR연동 (GoogleVisionAPI),공식·커스텀알레르기성분매칭로직 건강관리(피부·대변체크)-CRUD,Python분석서버연동 산책로추천·맞춤산책로추천로직	알레르기·건강관리(피부/대변체크)·산책·도메인프론트구현 전역테마·공통스타일구축(theme.css) 공통페이지메뉴·대시보드·랜딩페이지스타일링
이재원	팀원	홀캠안전구역이탈감자-YOLOv8n+OpenCV 울음소리분석·사전필터링+OllamaAI호출	감자마켓-CRUD·WebSocket채팅 홀캠안전관리·안전구역이탈감지로직 울음소리분석·울음소리분석로직구현 서비스배포환경-Docker·DockerCompose기반전체서비스컨테이너화	감자마켓(중고거래)·홀캠·울음소리분석 도메인프론트구현
임대한	팀원	AI맞춤동화생성-Ollama 동화음성생성-Supertonic+PiperTTS AndroidSMS실발송-OpenClaw+ PyThon/Termux 정부지원금RAG검색-ChromaDB	회원인증-JWTAccess·RefreshToken,Kakao계정연동 가족관리·가족초대·다중프로필 응급의료지원·공공API기반응급실병상조희· SOS추천 SMS미션·미션전송·비동기발송·상태관리	Kakao로그인·계정연동 가족관리·다중프로필 병원·지도·예약·대기 SOS·문자요청 맞춤동화·동화생성·TTS재생
윤승진	팀원	정부지원금후보선별·챗봇응답생성· AI모델연동 유사정부정책조화-ChromaDB임베딩기반RAG	퀘스트관리-CRUD·스케줄링 정부지원금추천·지원금후보추천로직 정부지원금챗봇·챗봇API구현	퀘스트·정부지원금어시스턴트·챗봇프론트구현

기술스택

AI & Data Engine

- Ollama(qwen3): 챗봇 · 분류 · 브리핑 · 상담 · 울음소리분석 · 산책
- OllamaVision(qwen2.5V): 피부 · 대변상태분석
- VARCOVISION-2.0-1.7B: 육아일기 AI 자동작성
- sentence-transformers: 리콜 유사도 매칭, 임베딩 엔진
- YOLOv8n+OpenCV: 홈캠 안전 구역 이탈 감지
- sklearn (LinearRegression/IsolationForest): 수면 패턴 분석
- moviepy+TTS(Supertonic/Piper): 육아일기 영상 · 동화 음성 생성
- RAG(ChromaDB+임베딩검색): 정부 지원금 데이터 벡터 검색 기반 추천

Frontend

- React(Vite): 컴포넌트 기반 SPA 구축
- TypeScript: 타입 안정성 기반 개발
- Redux Toolkit(RTK): 전역 상태 관리
- Axios: REST API 통신 및 JWT 인터셉터
- STOMP/SockJS: WebSocket 기반 실시간 채팅(베이비시터 · 마켓 · 퀘스트)
- Kakao Maps.JS SDK: 베이비시터, 감자마켓, 병원, 산책, 앨범
- Recharts/exifr: 성장 차트, 사진 EXIF 메타데이터 처리
- React.lazy/Suspense: 라우트 단위 코드 스플리팅

API & Infrastructure

- SafetyKorea Open API: 리콜 · 인증 정보 공공 API 연동
- 공공데이터포털 API: 응급실 · 기상 · 복지 서비스 정보 연동
- Kakao Login/Map/Local API: OAuth2.0 소셜 로그인, 지도, 좌표 검색, 앨범
- NAVER NEWS/YOUTUBE API: AI 행동 교정
- Google Vision API: 알라지 · 리콜 라벨 이미지 OCR
- FastAPI: AI 매칭 · 분석 전용 보조 API 서버
- OpenCraw+SMS: SMS 자동 발송, 일일 퀘스트
- GitHub: Git 기반 버전 관리 및 협업
- Docker/Docker Compose: 멀티 컨테이너 오케스트레이션
- AWS: EC2, RDS 등 클라우드 인프라 구축
- MariaDB: 관계형 데이터베이스

Backend

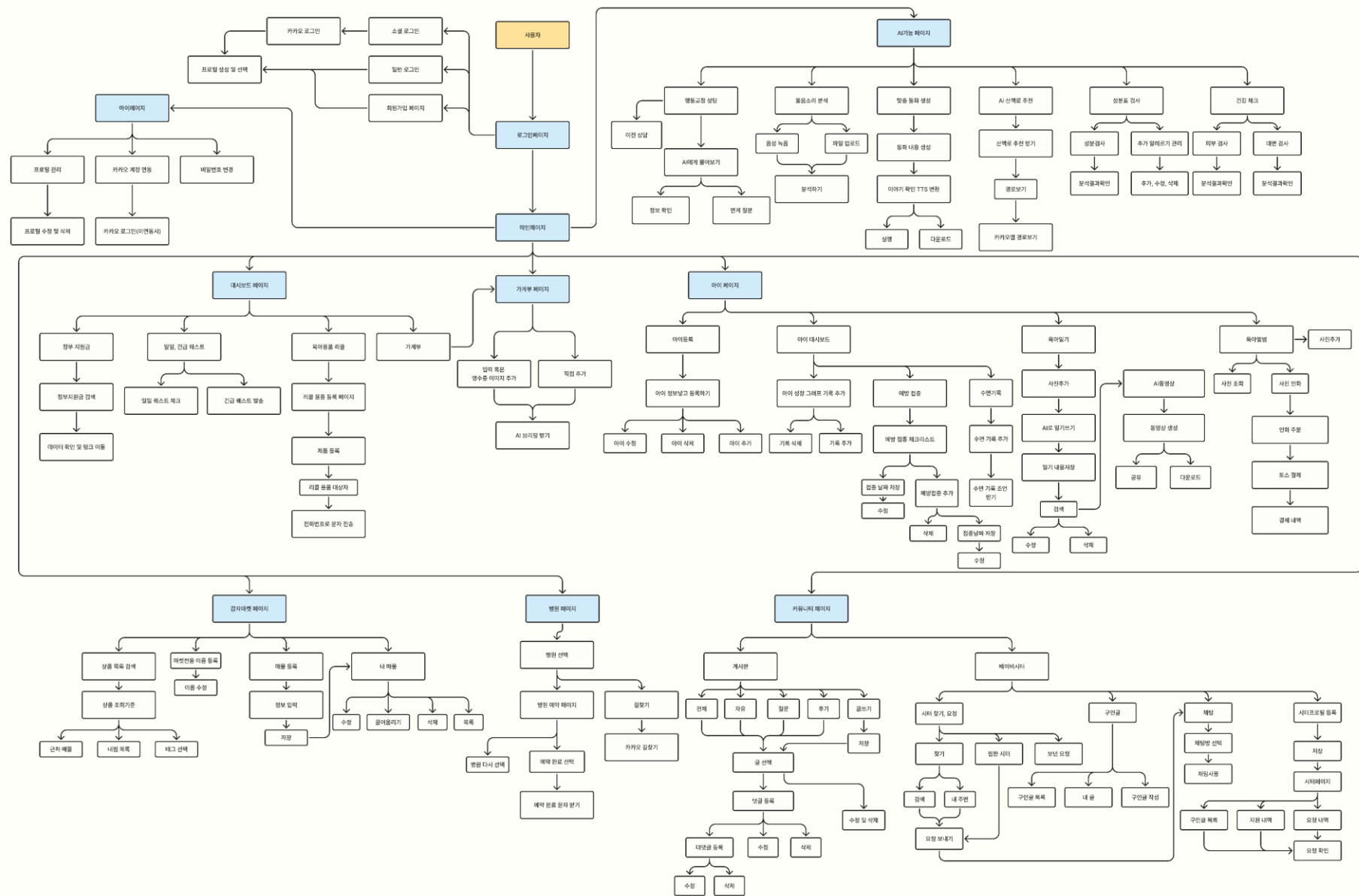
- Spring Boot 3.5: 비즈니스 로직 및 RESTful API 서비스
- Spring Security & JWT: Access/Refresh Token, Kakao OAuth 연동 인증
- MyBatis: SQL 매핑 기반 데이터 접근
- Spring AI: 가계부도 메인 LLM 연동(ChatClient)
- Caffeine: 좌표 기반 캐싱(Kakao API 호출 절감)

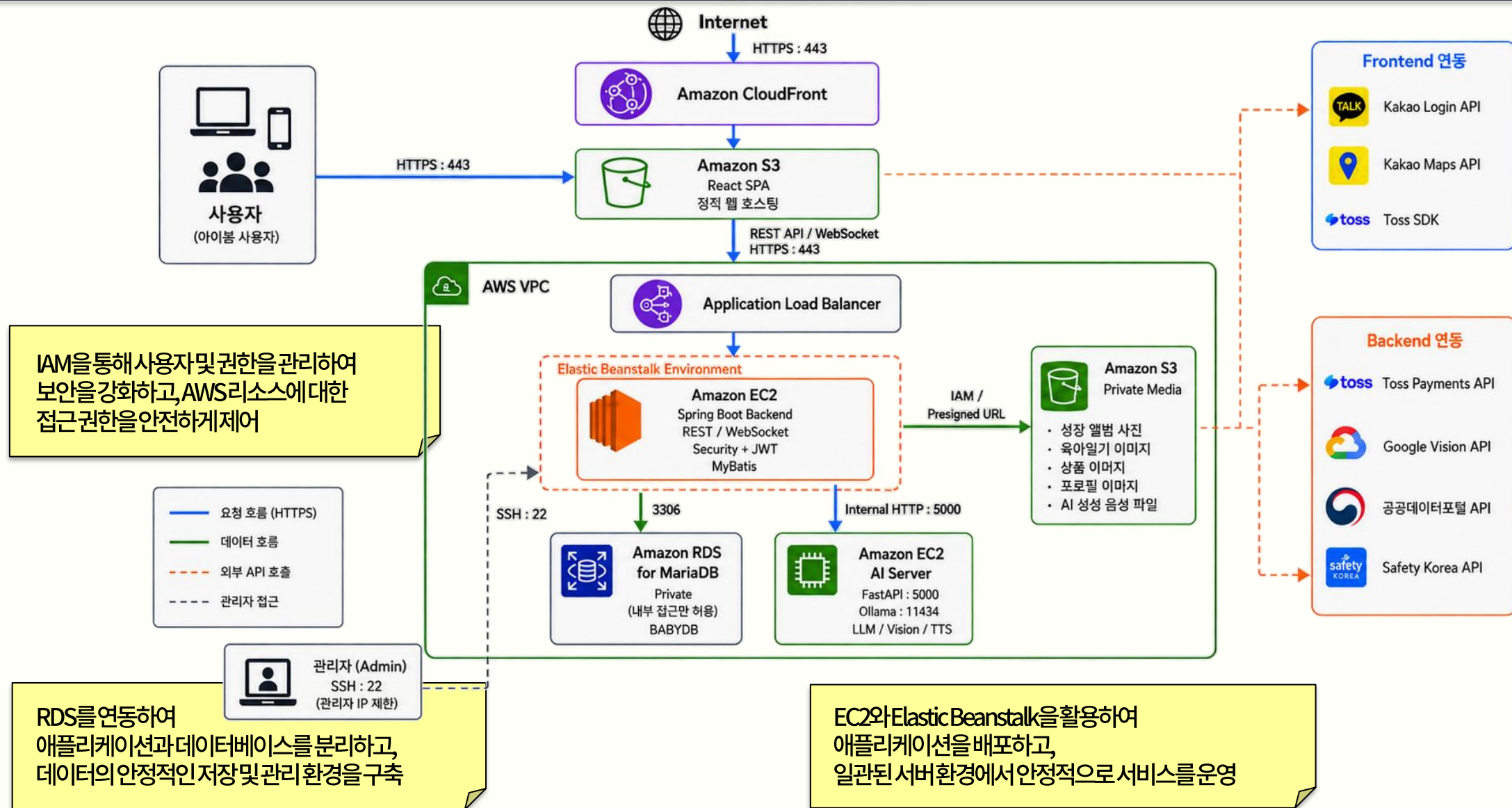
아이봄 프로젝트 WBS

기간: 2026-08-03 ~ 2026-08-30

구분	기능	시작일	종료일	8/3	8/4	8/5	8/6	8/7	8/8	8/9	8/10	8/11	8/12	8/13	8/14	8/15	8/16	8/17	8/18	8/19	8/20	8/21	8/22	8/23	8/24	8/25	8/26	8/27	8/28	8/29	8/30
공통	프로젝트 초기 세팅 (Git 전략, ERD 설계)	2026-08-03	2026-08-04																												
공통	Docker/WSL 통합 개발환경 구축 (5개 컨테이너)	2026-08-03	2026-08-04																												
공통	MyBatis 공통 규칙 정립 (스키마 네이밍, Mapper 등록)	2026-08-04	2026-08-05																												
공통	메인페이지 UI/UX 설계 (스토리텔링 구조 확정)	2026-08-10	2026-08-14																												
공통	메인페이지 공통 CSS 적용	2026-08-14	2026-08-17																												
공통	전체 기능 통합 테스트 (DB 초기화 후 순차 점검)	2026-08-18	2026-08-18																												
공통	OpenClaw 전화문자 자동화 인프라 연동	2026-08-20	2026-08-24																												
공통	통합 서버 실행 환경 안정화 (멀티 컨테이너)	2026-08-24	2026-08-25																												
공통	발표자료(PDF)-시연영상-README 최종 제작	2026-08-25	2026-08-28																												
황용현	아이 정보(BabyInfo) CRUD + 성장 기록	2026-08-04	2026-08-09																												
황용현	앨범 업로드(무한스크롤) + 인화주문 CRUD	2026-08-09	2026-08-13																												
황용현	예방접종 스케줄 + 수면기록 CRUD	2026-08-09	2026-08-14																												
황용현	관리자 페이지 (회원관리, 게시물 소프트삭제)	2026-08-11	2026-08-14																												
황용현	육아일기 CRUD + AI 자동일기 생성(VARCO-VISION)	2026-08-13	2026-08-19																												
황용현	AI 행동교정 상담 (뉴스-영상 연동 추천)	2026-08-19	2026-08-25																												
권용익	가계부 CRUD + 영수증 OCR 등록	2026-08-04	2026-08-11																												
권용익	가계부 AI 자동분류브리핑 (Spring AI)	2026-08-11	2026-08-17																												
권용익	리콜 제품 등록 + SafetyKorea API 연동	2026-08-06	2026-08-12																												
권용익	리콜 AI 유사도 매칭(Python) + SMS 알림	2026-08-12	2026-08-20																												
권용익	베이비시터 프로필/구인지원/폼/후기	2026-08-14	2026-08-20																												
권용익	베이비시터 실시간 채팅(WebSocket)	2026-08-20	2026-08-23																												
이민주	알리지 성분표 OCR(Google Vision) + 커스텀 등록	2026-08-05	2026-08-12																												
이민주	건강체크 (피부대변, Ollama Vision 분석)	2026-08-14	2026-08-20																												
이민주	산책로 추천 (카카오 로컬API + 날씨 연동)	2026-08-18	2026-08-23																												
이재원	감자마켓 매물 CRUD(등록/목록/지도) 백엔드	2026-08-06	2026-08-11																												
이재원	감자마켓 프론트 (목록상세-등록 화면)	2026-08-11	2026-08-14																												
이재원	감자마켓 찜 실시간채팅-거래완료 로직	2026-08-14	2026-08-19																												
이재원	홈 안전영역 설정 + YOLOv8 이탈감지	2026-08-19	2026-08-25																												
이재원	울음소리 분석 (온디바이스 피치검출 + Ollama)	2026-08-15	2026-08-21																												
이재원	전체 서비스 Docker Compose 컨테이너화	2026-08-24	2026-08-27																												
임대한	회원가입-카카오 소셜로그인-JWT 인증	2026-08-04	2026-08-08																												
임대한	가족 연결(부부 계정) + 프로필 구조	2026-08-08	2026-08-14																												
임대한	병원 예약 (대기현황-추천-문자알림)	2026-08-12	2026-08-20																												
임대한	AI 통화 생성 (TTS 연동)	2026-08-20	2026-08-26																												
임대한	OpenClaw SMS/전화 메시지 인프라 구축	2026-08-11	2026-08-21																												
윤승진	오늘의 퀘스트-긴급퀘스트 CRUD	2026-08-04	2026-08-09																												
윤승진	AI 챗봇 (프롬프트 응답) 1차 구현	2026-08-09	2026-08-14																												
윤승진	정부지원금 후보 추천 (공공데이터 연동)	2026-08-11	2026-08-18																												
윤승진	정부지원금 RAG (ChromaDB 벡터검색) 고도화	2026-08-18	2026-08-26																												
윤승진	챗봇 Function Calling 연동	2026-08-18	2026-08-21																												

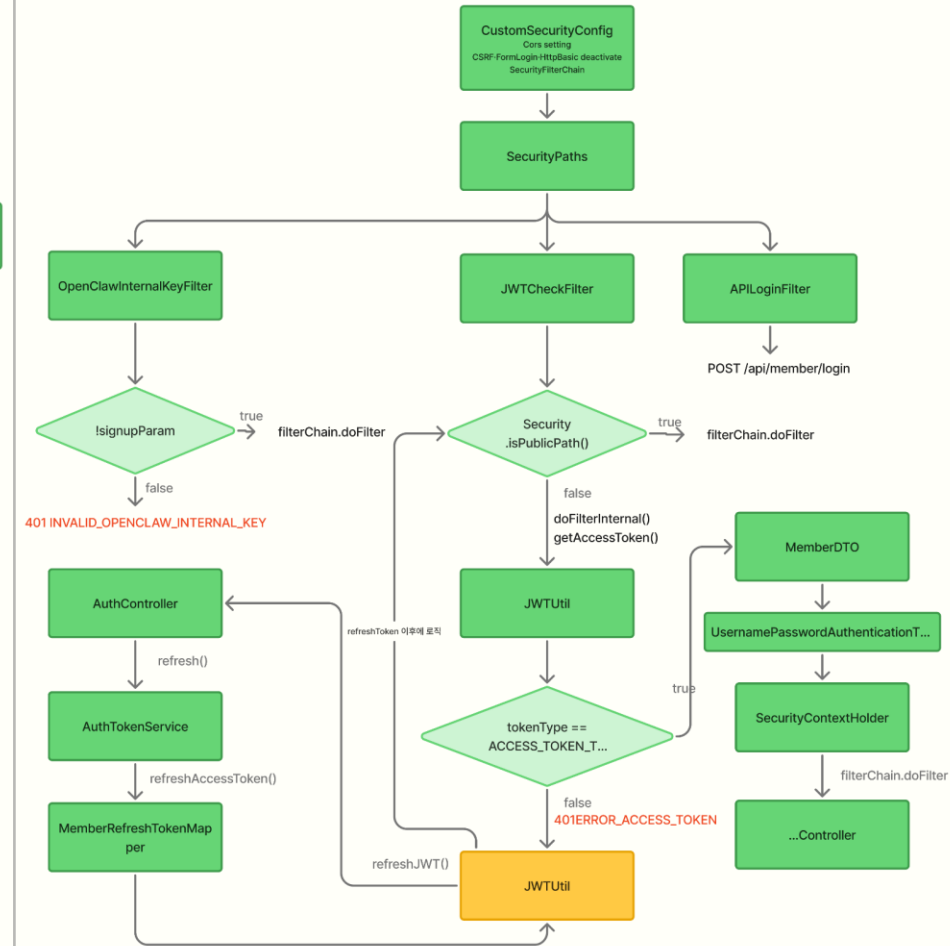
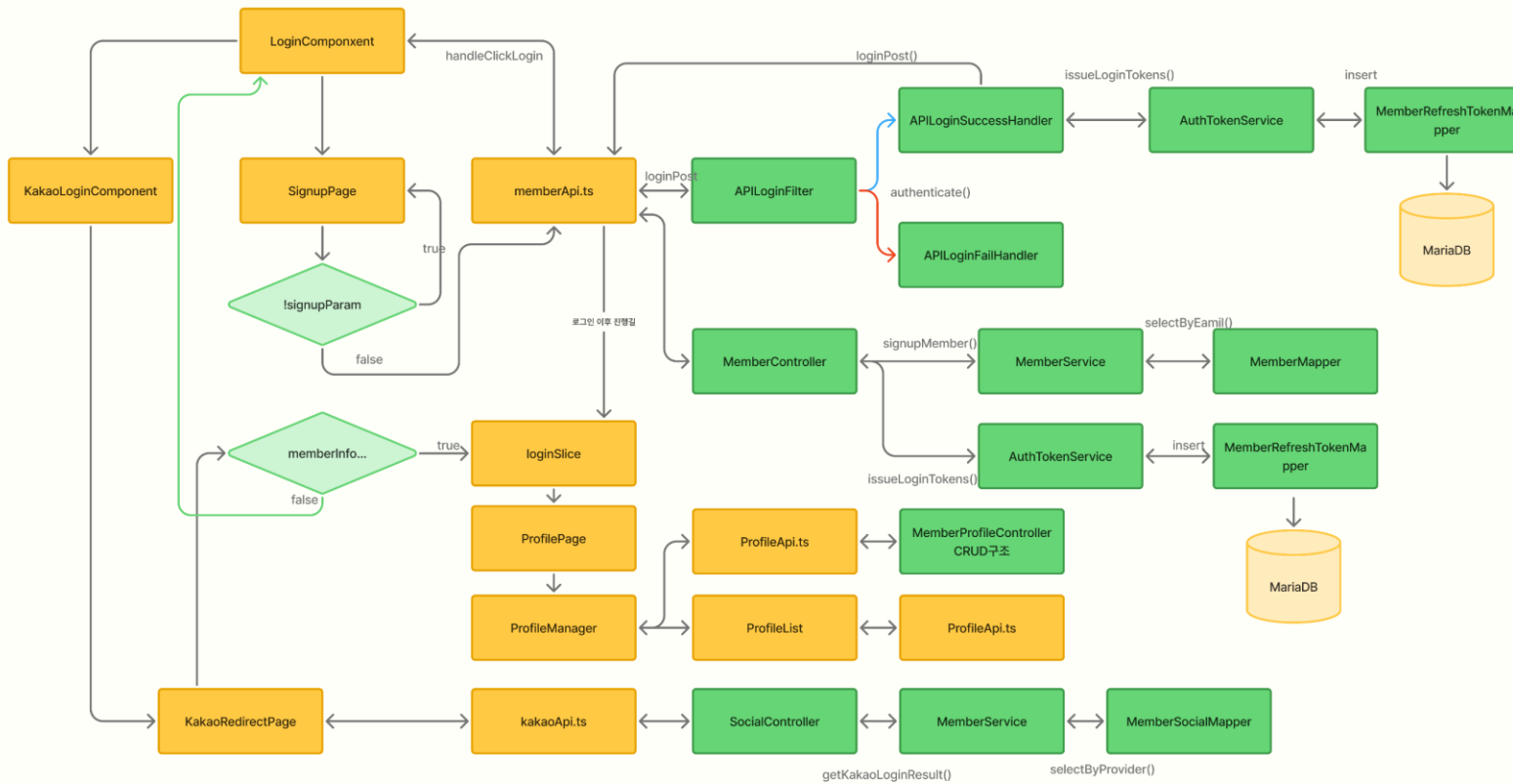
담당자 범례	
	공통
	황용현
	권용익
	이민주
	이재원
	임대한
	윤승진





Security + Login 플로우

JWT와 토큰 관리를 활용한 인증 · 인가 보안 구조



PYTHON
DATA BASE
BACK-END
FRONT-END
EXTERNAL-API

Security + Login

JWT와 토큰 관리를 활용한 인증 · 인가 보안 구조

주기능

- 로그인 성공시 Access · Refresh 분리발급
- JWT 검증후
사용자 권한에 따라 API 접근 처리
- 재사용 감지 및 로그아웃 시 해당 토큰 폐기



MEMBER LOGIN

로그인

이메일

비밀번호

로그인하기

처음이신가요? 회원가입

도움

K 카카오톡 계속하기

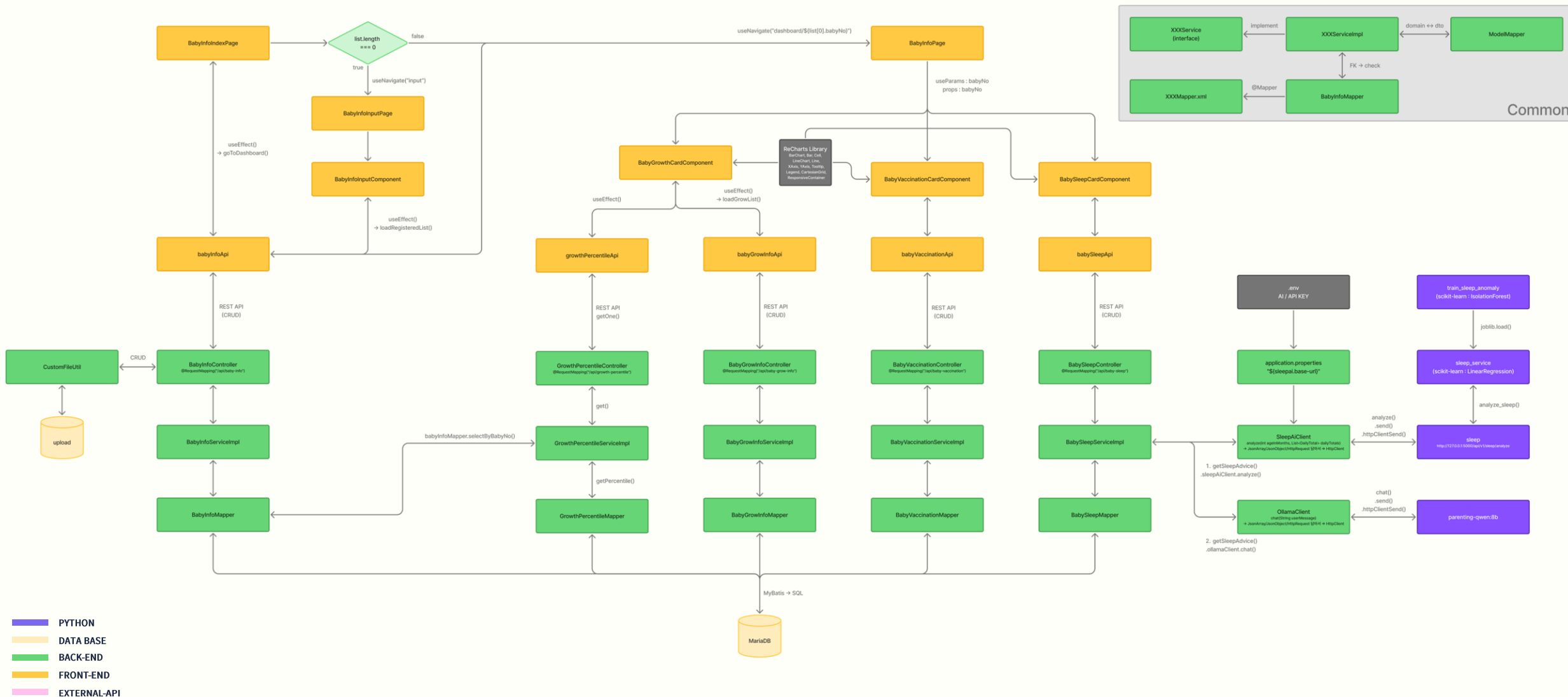
최초 로그인

```
String email = getValue(loginData, "email");
String pw = getValue(loginData, "pw");

UsernamePasswordAuthenticationToken authToken =
    new UsernamePasswordAuthenticationToken(email, pw);

return getAuthenticationManager()
    .authenticate(authToken);
```

아이정보는 props로 관리



아이정보등록및통계페이지

아이정보는 props로 관리

아이 정보 등록부터 성장, 예방접종, 수면기록까지 한 곳에서 관리하고 Recharts로 시각화하며 AI가 수면 패턴까지 분석해주는 대시보드

1 아이정보 확인
없으면 아이등록으로 이동

2 기본정보 CRUD
아이등록·수정·삭제

3 성장 데이터 시각화
Recharts 기반 그래프
성장·접종·수면 기록 CRUD

4 AI 수면 분석
파이썬 AI 조언

1. 아이정보가 필요한 페이지 이동

localhost:3000 내용:

등록된 아이가 없습니다. 먼저 아이를 등록해주세요.

확인

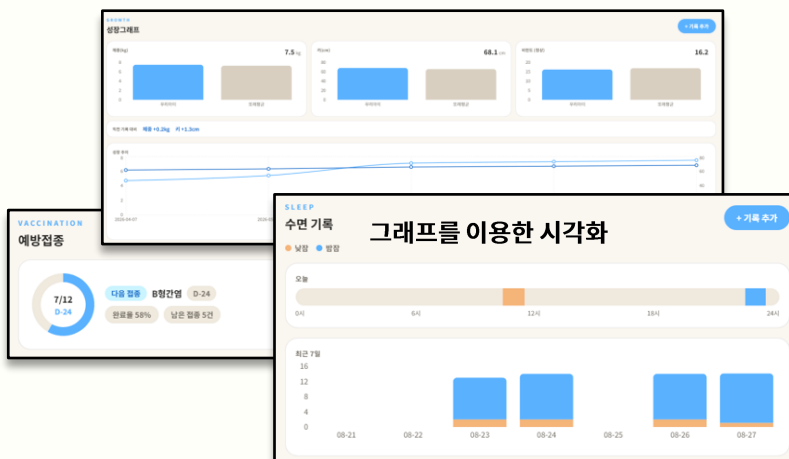
아이 등록하기

- 아이 정보는 최상단 Page에서 props로 내려줌
- 아이 정보 유무 체크 -> 없으면 아이등록 페이지로 이동

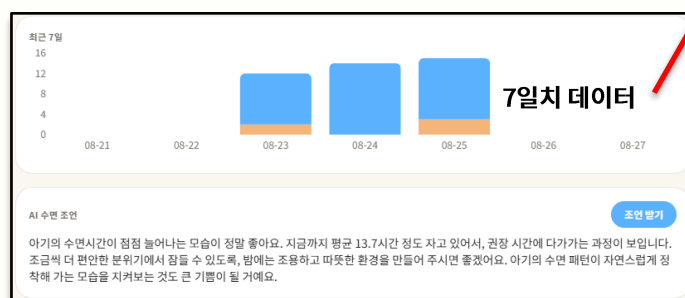
```
const list: BabyInfo[] = await babyInfoApi.getList();

if (list.length === 0) {
  alert("등록된 아이가 없습니다. 먼저 아이를 등록해주세요.");
  navigate("input", { replace: true });
  return;
}
```

2. Recharts를 이용한 데이터 시각화 및 CRUD



3. 파이썬 AI를 활용한 최근 7일 수면 패턴 분석



최근 7일 수면기록을 TreeMap으로 날짜순으로 집계
HashSet으로 기록 여부를 표시 -> List로 합쳐 파이썬 API로 전달

```
Map<LocalDate, int[]> dailyMinutes = new TreeMap<>(); // 날짜별 수면시간 집계
Set<LocalDate> recordedDates = new HashSet<>(); // 기록 있는 날짜 표시

for (BabySleep sleep : allSleep) {
  LocalDate date = sleep.getStartTime().toLocalDate();
  recordedDates.add(date);

  LocalDateTime end = sleep.getEndTime() != null ? sleep.getEndTime() :
    LocalDateTime.now();
  long minutes = Duration.between(sleep.getStartTime(), end).toMinutes();

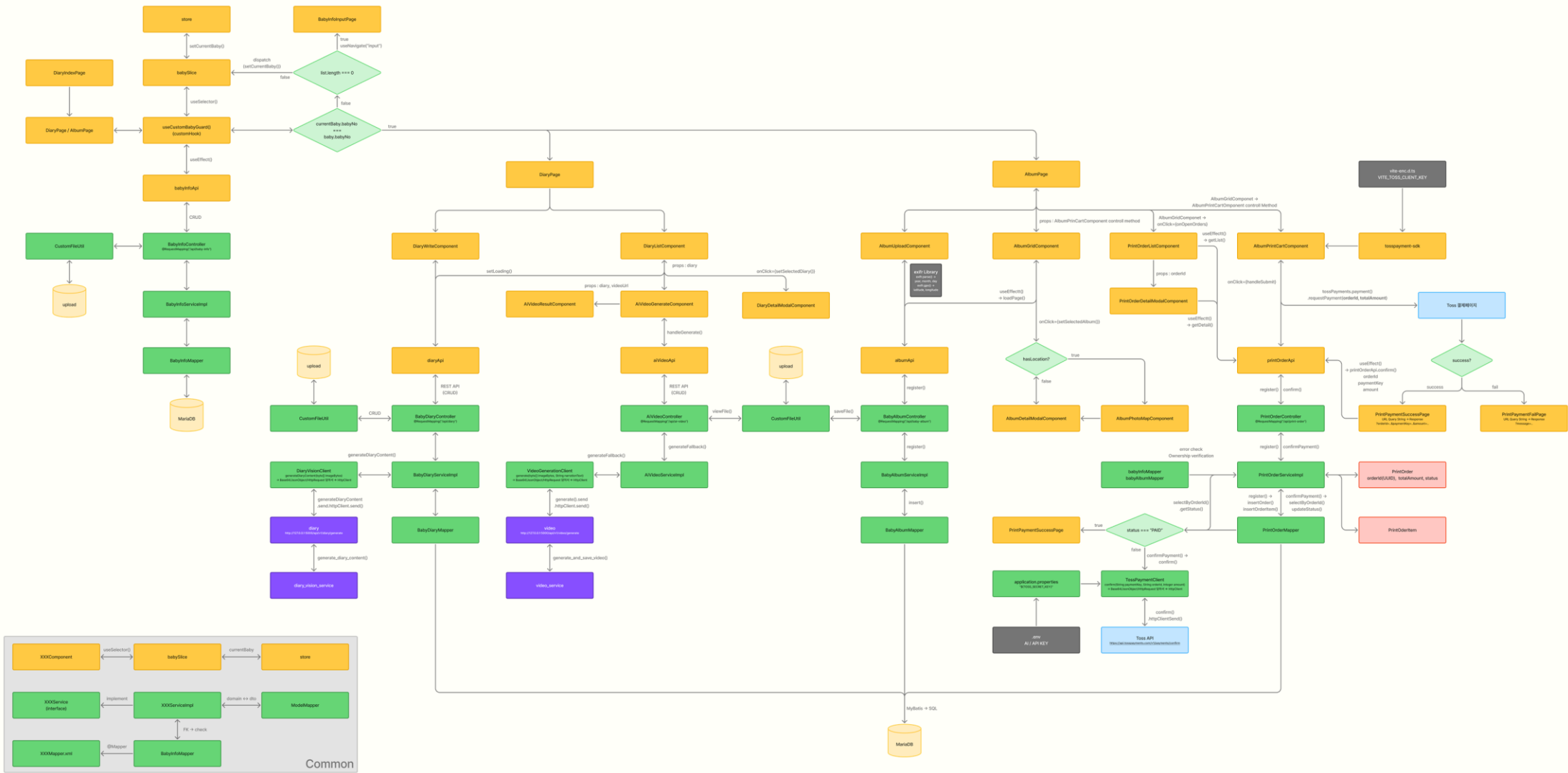
  int[] totals = dailyMinutes.get(date);
  if ("낮잠".equals(sleep.getSleepType())) totals[0] += (int) minutes; // 낮잠
  else totals[1] += (int) minutes; // 밤잠
}

List<SleepAiClient.DailyTotal> dailyTotals = new ArrayList<>();
for (var e : dailyMinutes.entrySet())
  dailyTotals.add(new SleepAiClient.DailyTotal(e.getKey().toString(),
    e.getValue()[0], e.getValue()[1], recordedDates.contains(e.getKey())));

sleepAiClient.analyze((int) ageInMonths, dailyTotals); // Python AI 서버로 전달
```

육아일기 및 앨범 플로우

아이 정보는 RTK(Redux Toolkit)로 전역 관리



- PYTHON
- DATA BASE
- BACK-END
- FRONT-END
- EXTERNAL-API

육아일기를 직접 또는 AI로 작성하고 검색 및 페이지징으로 지난 기록을 찾아보며 Python을 이용하여 영상을 만들어 주는 페이지

1. 사진을 넣어 AI가 육아일기를 생성



```
// BabyDiaryServiceImpl.java - MultipartFile -> byte[]
byte[] imageBytes = image.getBytes();
return diaryVisionClient.generateDiaryContent(imageBytes);

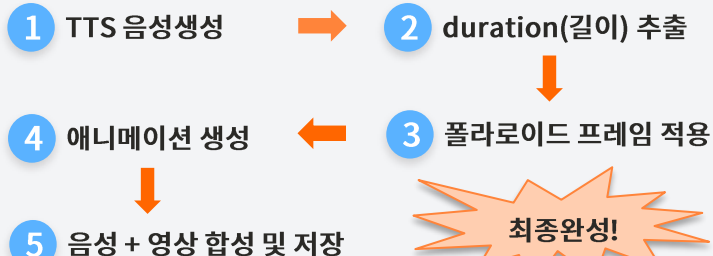
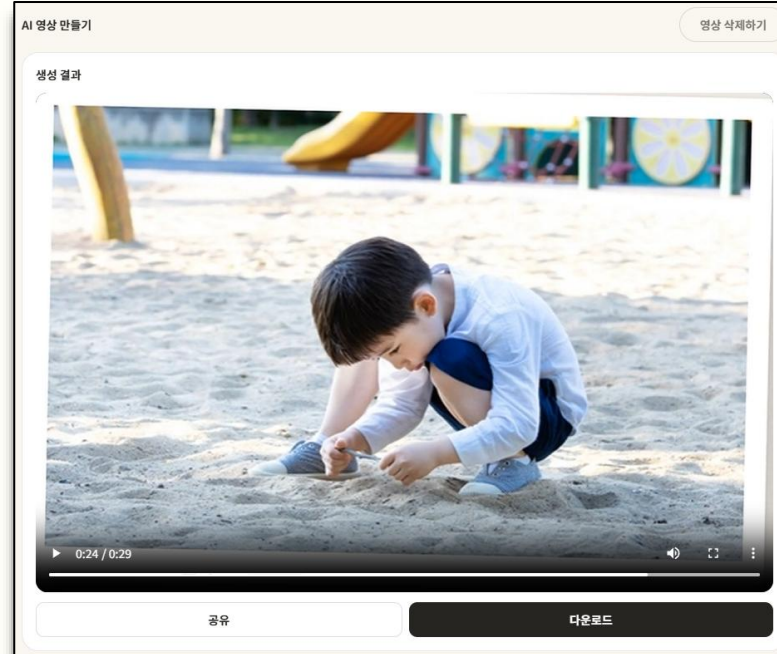
// DiaryVisionClient.java - byte[] -> Base64 -> HTTP 전송
String base64Image = Base64.getEncoder().encodeToString(imageBytes);
JsonObject body = new JsonObject();
body.addProperty("imageBase64", base64Image);
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create(baseUrl + "/api/v1/diary/generate"))
    .POST(HttpRequest.BodyPublishers.ofString(body.toString()))
    .build();
```

2. 육아일기를 검색 기능 및 페이지징 처리



```
<select id="selectList" resultType="com.backend.diary.domain.BabyDiary">
    SELECT bd.diary_no, bd.baby_no, bd.diary_date, bd.photo_file_name,
    bd.content, bd.reg_time, bd.mod_time
    FROM tbl_baby_diary bd
    JOIN tbl_baby_info bi ON bd.baby_no = bi.baby_no
    WHERE bd.baby_no = #{babyNo} AND bi.email = #{email}
    <if test="keyword != null and keyword != ''">
        AND bd.content LIKE CONCAT('%', #{keyword}, '%')
    </if>
    ORDER BY bd.diary_date DESC, bd.diary_no DESC
    LIMIT #{size} OFFSET #{skip}
</select>
```

3. 작성된 육아일기를 동영상으로 변환



```
def compose_diary_video(
    image: Image.Image,
    narration_text: str,
    tts_service: NarrationTtsService,
    output_path: str,
    max_dimension: int = 1280,
) -> float:
    """폴라로이드 프레임 + 애니메이션 + TTS 내레이션을 합성해서 mp4로 저장한다."""
    1 waveform, sample_rate = tts_service.synthesize(narration_text)

    wav_path = tempfile.mktemp(suffix=".wav")
    scipy.io.wavfile.write(wav_path, sample_rate, waveform)

    audio_clip = AudioFileClip(wav_path)
    2 duration = audio_clip.duration

    3 framed = add_polaroid_frame(image)
    ratio = framed.width / framed.height
    if ratio >= 1:
        output_size = (max_dimension, round(max_dimension / ratio))
    else:
        output_size = (round(max_dimension * ratio), max_dimension)

    4 video_clip = create_kin_burns_clip(framed, duration=duration,
        output_size=output_size)

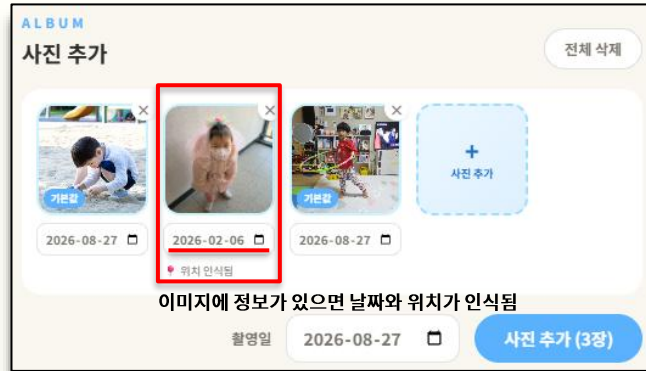
    5 final_clip = video_clip.with_audio(audio_clip)
    final_clip.write_videofile(output_path, fps=24, codec="libx264",
        audio_codec="aac", audio_fps=sample_rate)

    os.remove(wav_path)

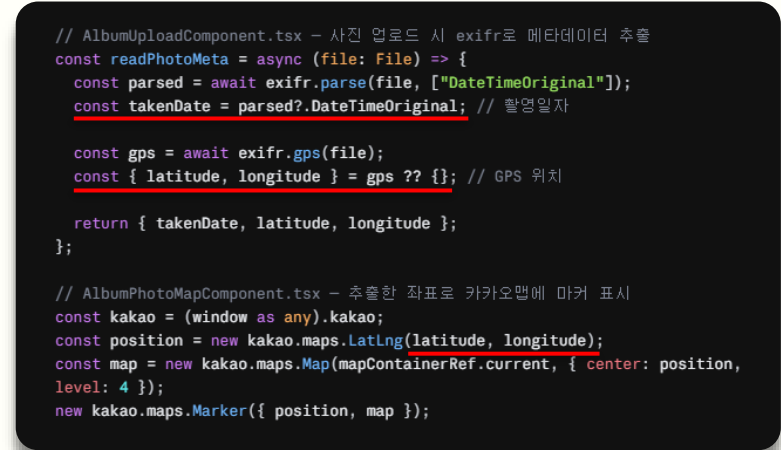
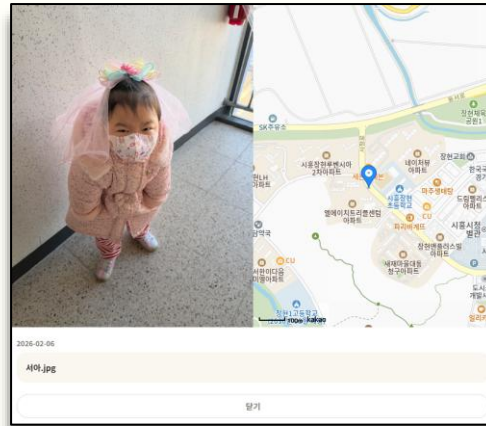
    return duration
```

드래그 앤 드랍으로 사진을 저장하고, **exifr**로 촬영일자, 위치정보 추출하며, **TossPayment**를 사용한 결제시스템을 구현한 페이지

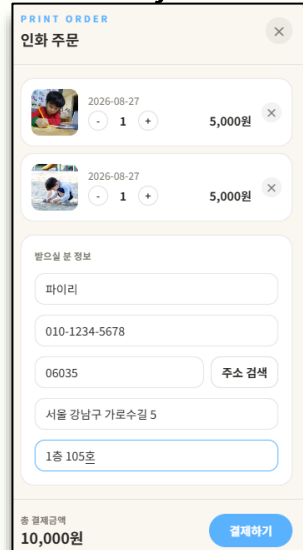
1. exifr를 이용한 이미지 데이터 추출



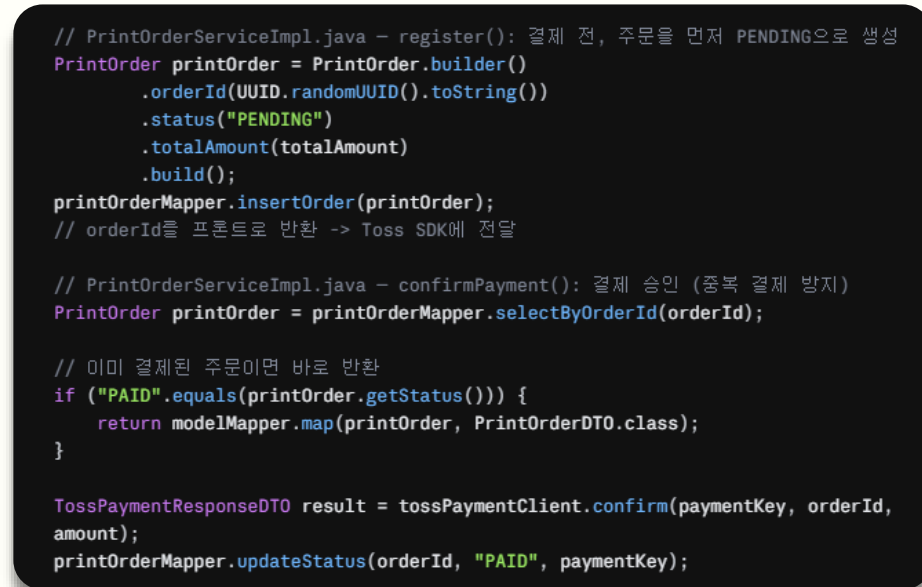
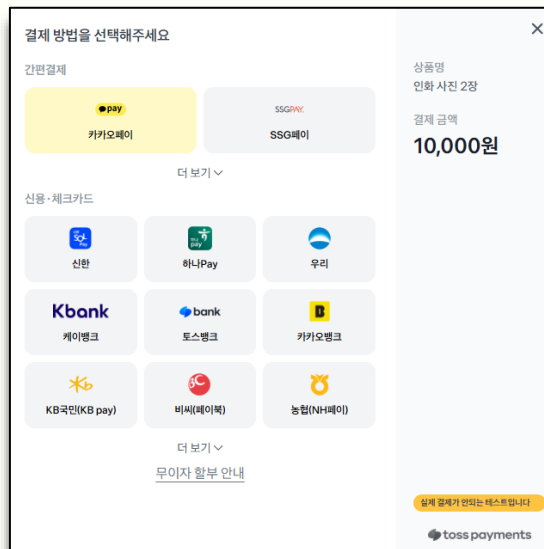
상세페이지



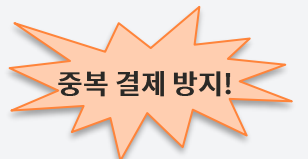
2. TossPayment를 이용한 결제



결제

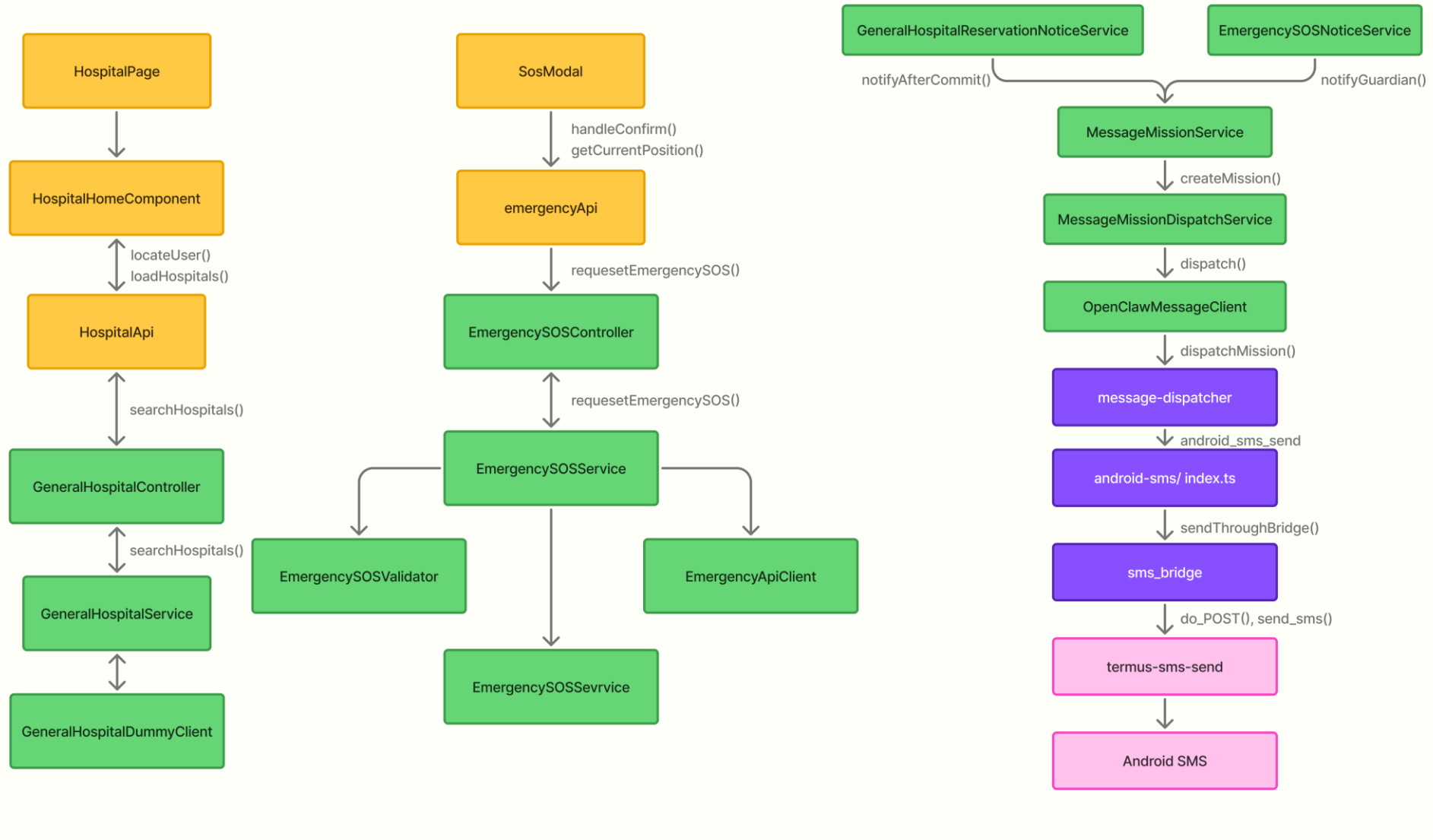


- 1 결제 전 백엔드 주문 생성
- 2 주문의 orderId를 Toss SDK에 전달
- 3 결제 성공시 상태가 "PENDING" -> "PAID" 변경
- 4 새로고침 눌러도 상태가 "PAID"이면 Toss API 전달 안함



Hospital+SOS Messaging플로우

병원 검색부터 SOS 문자 발송까지의 통합 처리 흐름



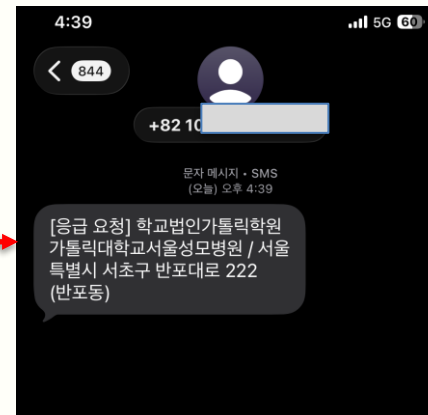
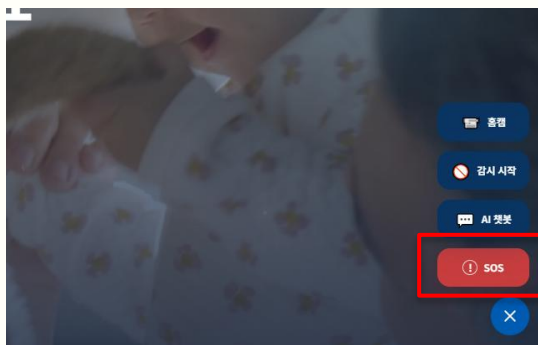
Hospital + SOS Messaging

병원 검색부터 SOS 문자 발송까지의 통합 처리 흐름

병원 검색 > 예약 요청 > 예약 정보 검증 > 예약 저장 > 보호자 알림 전송



SOS 요청 버튼 > 주위 응급실 조회 및 요청 > 보호자 알림 전송



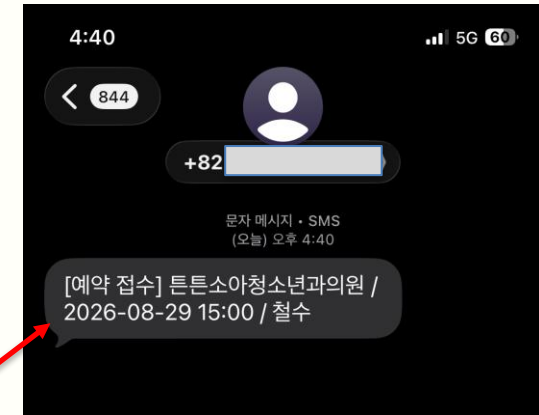
보호자 연락처
문자 발송 번호
010317
예약 확인 문자가 이 번호로 발송됩니다.

진료 메모
방문 사유
감기가 심하고 열이 조금 있어요

환소 질환
X

알려지
도마도

취소 예약 완료



문자데이터구조

Source=문자타입(병원예약, SOS, 리콜등)
To=보호자전화번호
Context=병원명/예약날짜등문자내용

위구조를통해재활용해서다른파트에서
사용할수있도록하였다

```
private void notifyguardian(
    GeneralHospitalReservation reservation) {
    MessageRequestDTO request =
        MessageRequestDTO.builder()
            .source(
                MissionSource.HOSPITAL_RESERVATION)
            .to(
                reservation.getNotificationPhone())
            .content(
                guardianNoticeContent(reservation))
            .build();

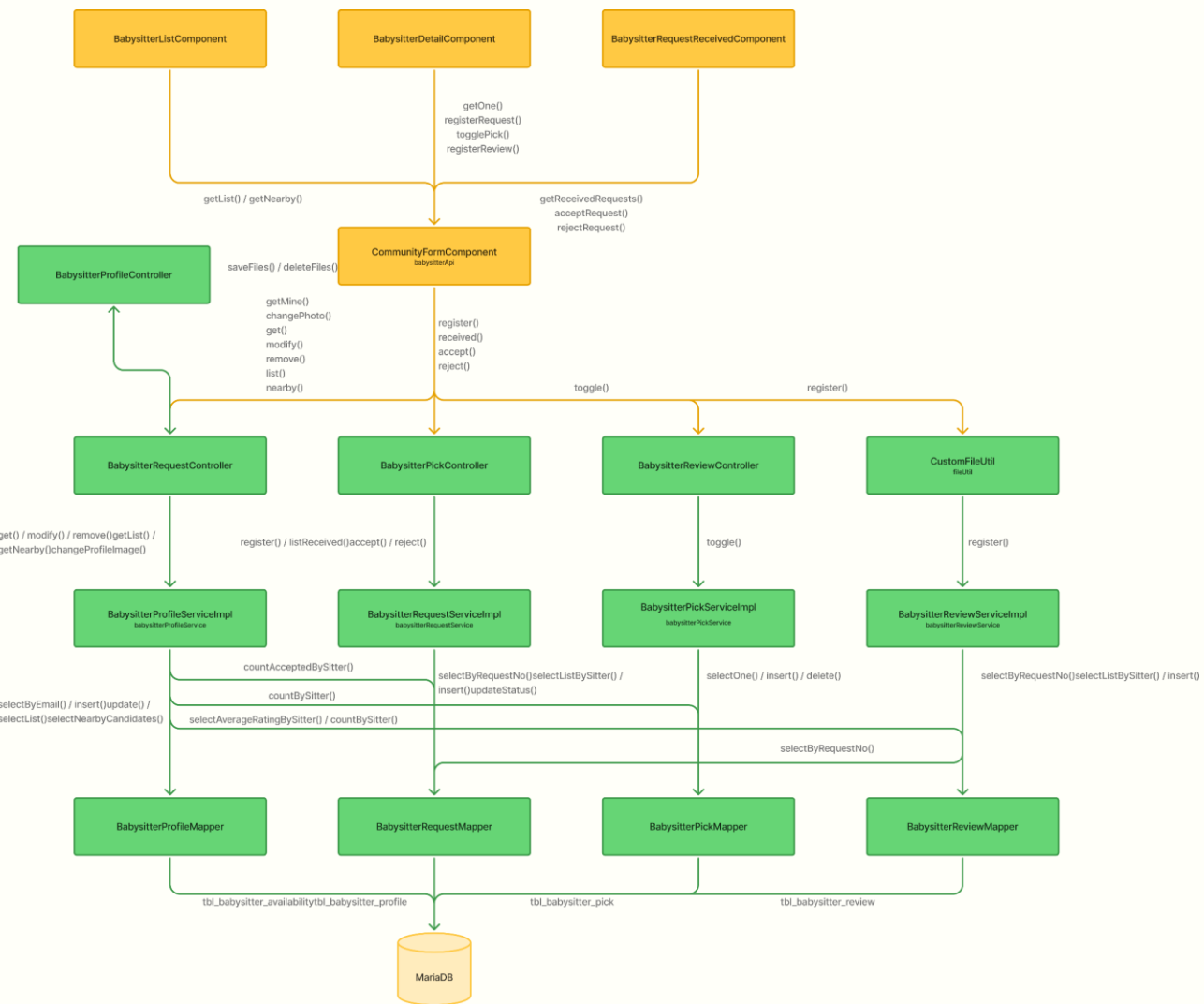
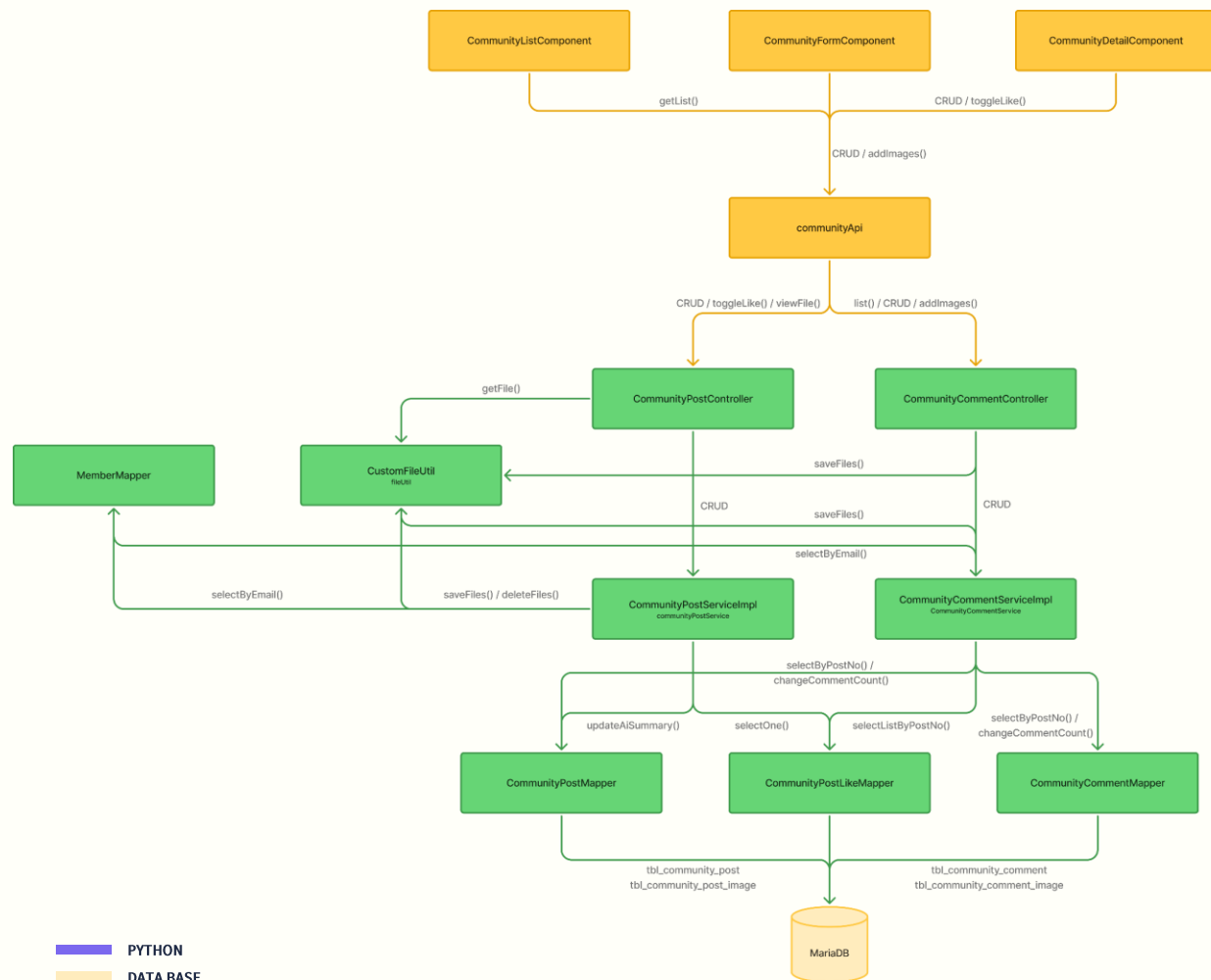
    MessageMissionDTO mission =
        messageMissionService.createMission(
            request,
            reservation.getMemberEmail()
        );

    messageMissionDispatchService
        .dispatch(mission);
}
```

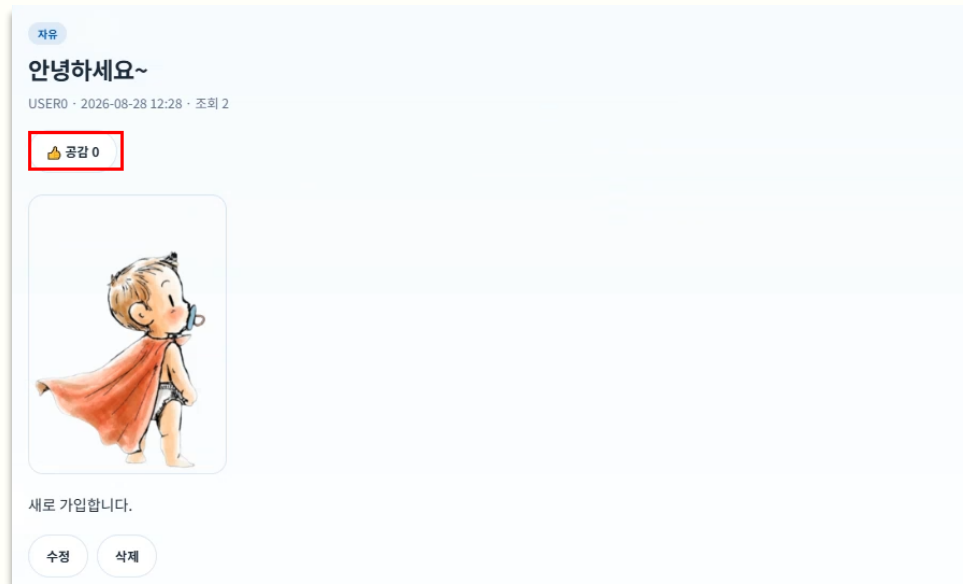
```
private String guardianNoticeContent(
    GeneralHospitalReservation reservation) {
    return "[예약 접수] "
        + reservation.getHospitalName()
        + " / "
        + reservation.getReservationDate()
        + " "
        + reservation.getReservationTime()
        + " / "
        + reservation.getPatientName();
}
```

커뮤니티 & 베이비시터 플로우

공감 기능과 WebSocket 기반 실시간 소통 구조



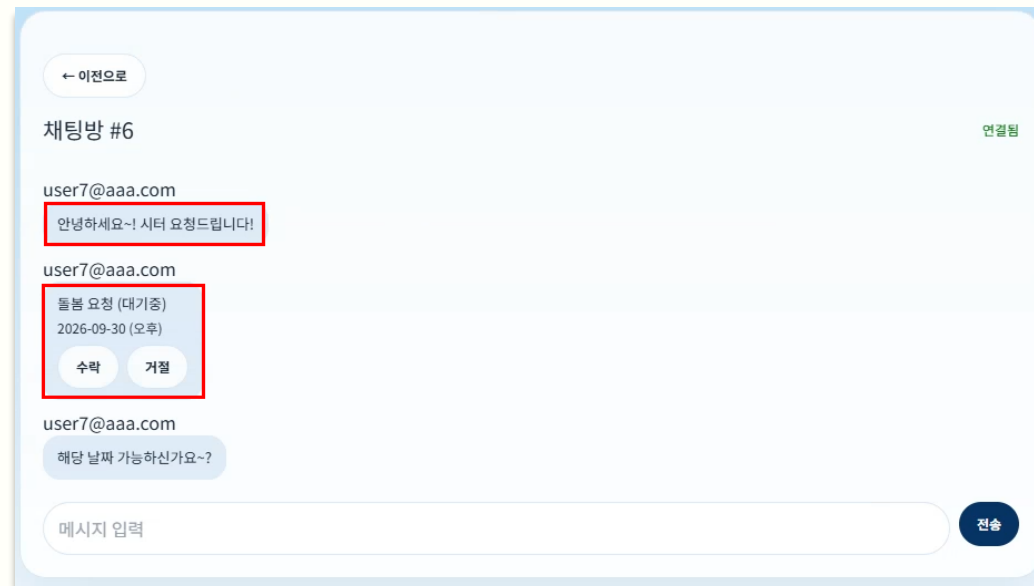
1. 커뮤니티 - 공감(좋아요) 토글



CommunityPostServiceImpl.toggleLike()

```
public boolean toggleLike(Long postNo, String memberEmail) {
    CommunityPostLike existing = communityPostLikeMapper
        .selectOne(postNo, memberEmail);
    if (existing != null) {
        communityPostLikeMapper.delete(postNo, memberEmail);
        communityPostMapper.changeLikeCount(postNo, -1);
        return false;
    }
    communityPostLikeMapper.insert(CommunityPostLike.builder()
        .postNo(postNo).memberEmail(memberEmail).build());
    communityPostMapper.changeLikeCount(postNo, 1);
    return true;
}
```

2. 베이비시터 - 실시간 채팅 (WebSocket)

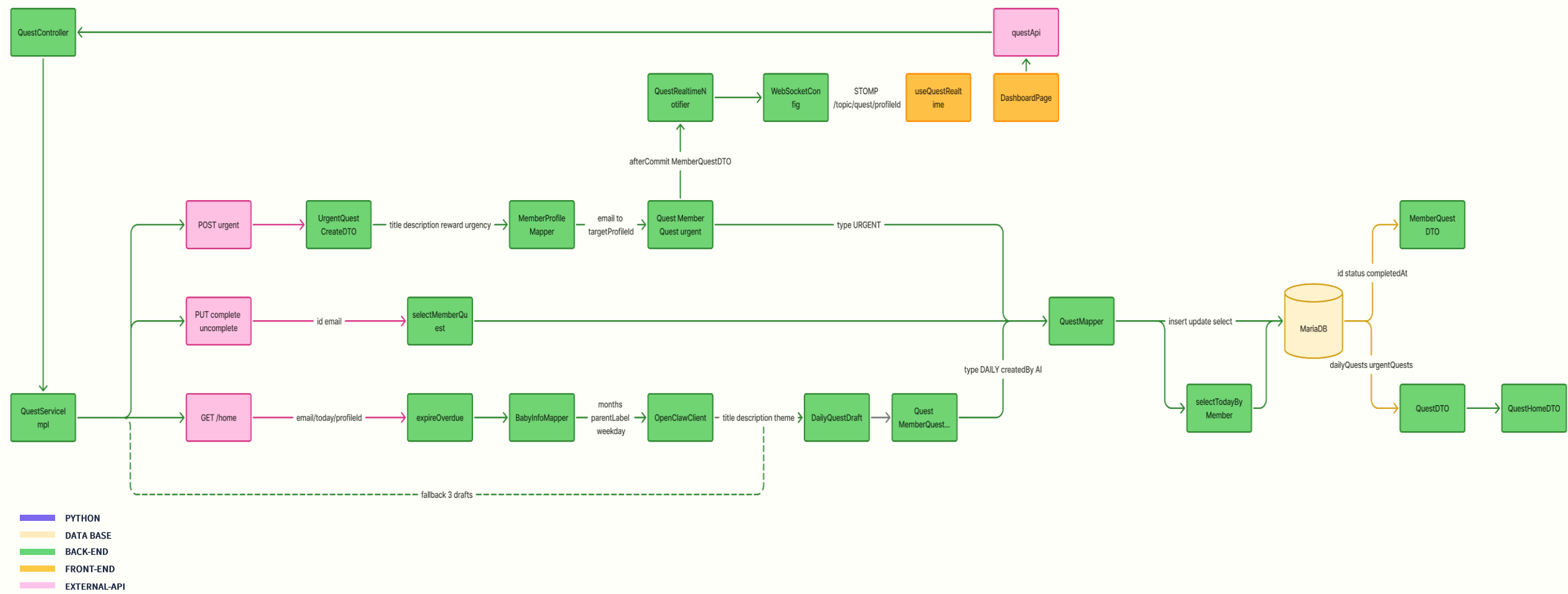


BabysitterChatWebSocketController

```
@MessageMapping("/babysitter-chat/{roomNo}/send")
@SendTo("/topic/babysitter-chat/{roomNo}")
public BabysitterChatMessageDTO send(
    @DestinationVariable Long roomNo,
    BabysitterChatMessageDTO chatMessageDTO) {
    chatMessageDTO.setRoomNo(roomNo);
    return babysitterChatMessageService
        .sendMessage(chatMessageDTO);
}
```


일일 퀘스트 플로우

같은계정의 다른 보호자에게 실시간 퀘스트 푸시가능



일일 퀘스트

같은계정의 다른 보호자에게 실시간 퀘스트 푸시가능

오늘의 할 일

0 / 3 완료

상대에게 긴급 할 일 보내기

예: 기저귀 사다 주세요

설명 (선택)

보내기

상대 프로필에 보냈습니다.

오늘의 할 일

0 / 3 완료

긴급

아이에게 점심을 주세요

확인이 필요한 긴급 할 일이에요.

완료

긴급

긴급퀘스트입니다.

보내기 버튼 누르면 상대방에게 긴급퀘스트 생성됩니다.

완료

상대에게 긴급 할 일 보내기

예: 기저귀 사다 주세요

설명 (선택)

보내기

오늘의 할 일

0 / 3 완료

상대에게 긴급 할 일 보내기

예: 기저귀 사다 주세요

설명 (선택)

보내기

기저귀 교대하기

10P

바닥에 앉아 놀이하기

10P

물 쥘겨주기

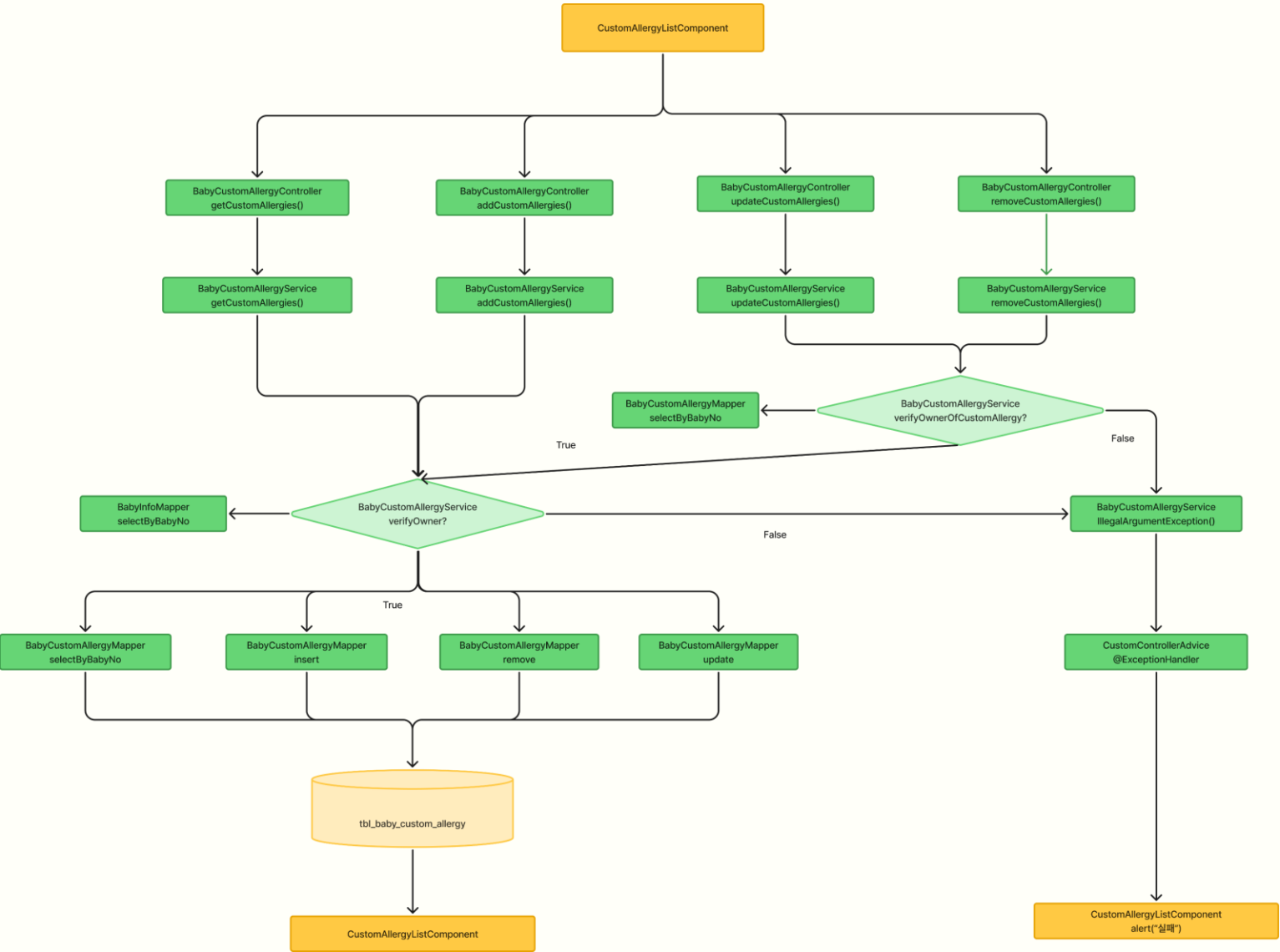
10P

```
MemberQuestDTO result = toDTO(questMapper.selectMemberQuest(mq.getId(), creatorEmail));  
questRealtimeNotifier.notifyUrgentAssigned(targetProfileId, result);  
return result;
```

- 상대가 같은 아이디의 다른 프로필을 쓰고 있으면, 긴급퀘를 만들었을 때 상대 화면에 바로 푸시된다. 서로 긴급 할 일을 주고받을 수 있다.
- 상대는 긴급 퀘스트를 받을 수 있고, 완료 버튼을 누르면 해당 퀘스트는 목록에서 자동으로 사라진다.
- 긴급퀘스트를 보내면 하단 창에 안내 메시지가 뜨고, 상대 프로필에 보냈다는 표시가 나온다.

```
private void assignBabyDailyQuests( 1개 사용 위치 8 DESKTOP-OT238HQ#EZEN  
    String email, Long profileId, String parentType, BabyInfo baby, LocalDate today) {  
    int months = ageInMonths(baby.getBirthDate(), today);  
    String weekday = today.getDayOfWeek().getDisplayName(TextStyle.FULL, Locale.KOREAN);  
    List<DailyQuestDraft> drafts = openClawClient.generateDailyQuests(  
        months, parentLabel(parentType), weekday);  
    if (drafts == null || drafts.size() != 3) {  
        drafts = fallbackDailyQuests(months, profileId, today);  
    }  
    return List.of(  
        DailyQuestDraft.builder().title(care[rot]).theme("CARE").description(" ").build(),  
        DailyQuestDraft.builder().title(activity[rot]).theme("ACTIVITY").description(" ").build(),  
        DailyQuestDraft.builder().title(chore[rot]).theme("CHORE").description(" ").build()  
    );
```

- 아이의 정보를 아직 등록하지 않았어도, 홈 ‘오늘의 할 일’에 퀘스트가 자동으로 생긴다.
- 아이 정보가 없어도 바로 쓸 수 있도록 기본 할 일 3개를 채워 둔다.
- 아이 정보를 등록하고 퀘스트 화면으로 돌아오면, 이전에 떠 있던 할 일은 목록에서 사라진다. 등록 직후 잠깐 빈 화면처럼 보일 수 있다.
- 지워진 자리에는 아이 정보(월령, 담당 부모, 요일)를 읽어서, 그 아이에게 맞는 할 일 3개가 다시 만들어진다.



알레르기 관리

알레르기 관리 - 수정·삭제 전 서버에서 소유자 검증

성분마다 존재 여부와 소유자를 먼저 검증하고, 수정·삭제가 끝나면 항상 서버에서 다시 불러와 **데이터 정합성**을 지키는 알레르기 성분 관리 흐름



1. 알레르기 성분 확인

```
private void verifyOwnerOfCustomAllergy(Long customAllergyNo, String email) {
    BabyCustomAllergy existing = babyCustomAllergyMapper.selectByCustomAllergyNo(customAllergyNo);
    if (existing == null) {
        throw new IllegalArgumentException("존재하지 않는 알레르기 성분입니다: " + customAllergyNo);
    }
    verifyOwner(existing.getBabyNo(), email);
}
```

verifyOwnerOfCustomAllergy — 수정하거나 삭제하려는 알레르기 성분이 실제로 존재하는지 먼저 확인합니다. 성분이 존재하면 해당 성분이 내 아이의 정보가 맞는지도 확인한 후 수정·삭제를 진행합니다.

● 존재하지 않는 성분이나 다른 아이의 성분을 수정·삭제하지 못하도록 확인합니다.

2. 수정 후 최신 정보 다시 불러오기

```
const handleUpdate = async (customAllergyNo?: number) => {
    if (!customAllergyNo || !editingName.trim()) return;

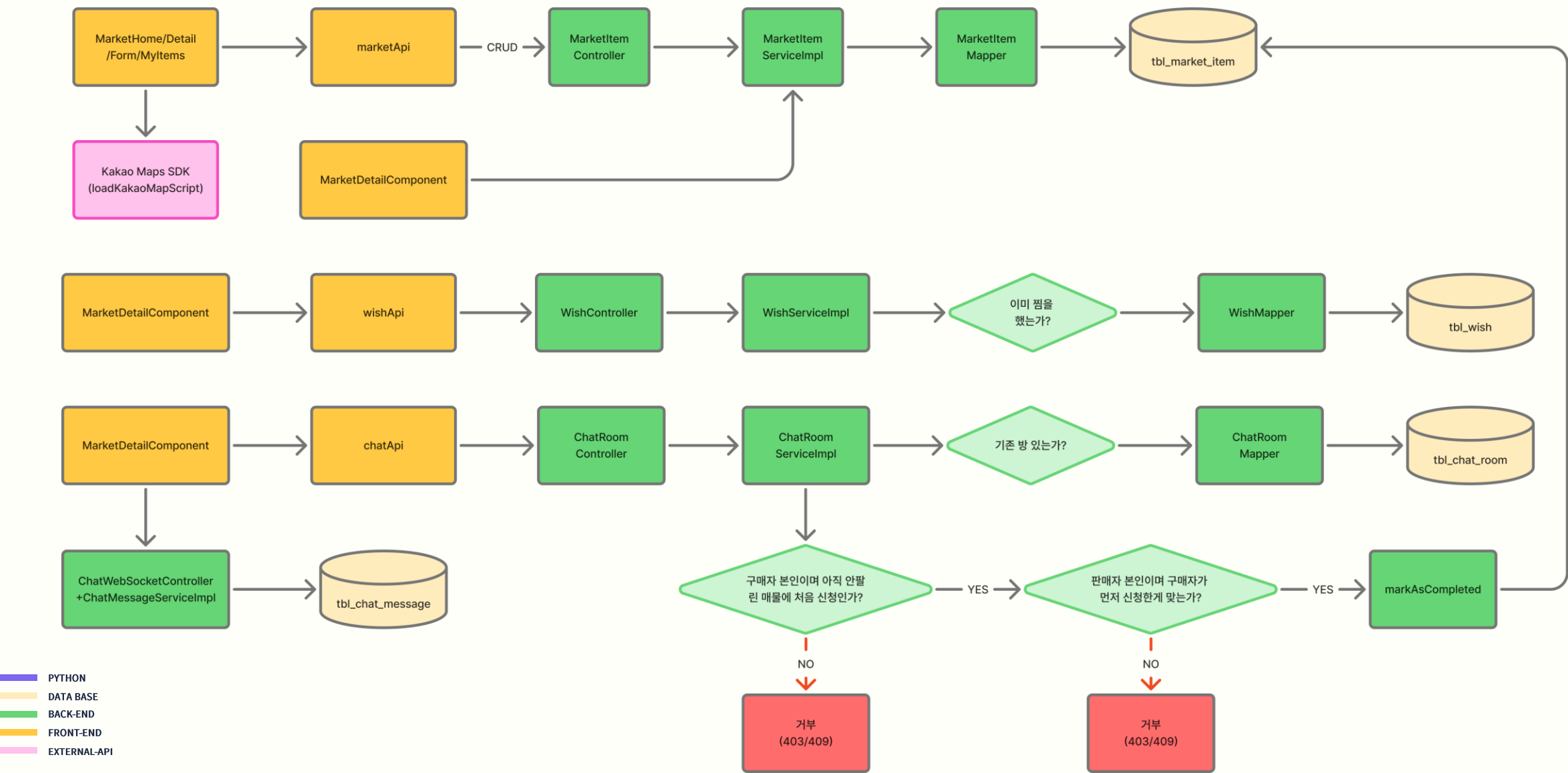
    try {
        await allergyApi.updateCustomAllergy(customAllergyNo, editingName.trim());
        cancelEdit();
        await load();
    } catch (err) {
        alert("수정에 실패했습니다.");
        console.error(err);
    }
}
```

handleUpdate → **load()** — 알레르기 성분 수정이 성공하면 수정 화면을 닫고 서버에 저장된 알레르기 성분 목록을 다시 불러옵니다. 화면에 기존 정보를 그대로 두지 않고 서버에서 최신 정보를 다시 받아오기 때문에, 실제로 저장된 내용과 화면에 보이는 내용을 일치시킬 수 있습니다.

● 수정이 완료되면 서버의 최신 정보를 다시 불러와 화면에 반영합니다.

감자마켓 Flow

거래완료는 신청 -> 확정 2단계 체인으로 강제



거래완료 · 온도 평가 완료

안녕하세요 포대기 아직 판매중인가요? 살게요!!

08/27 17:38

네 가능해요! 대우 효령아파트 105동 입구에서 20시 30분에 거래하는거 어떠세요?

08/27 17:38

네 좋아요 그때 빌게요~~

08/27 17:38

거래가 완료되어 더 이상 메시지를 보낼 수 없어요.

```
if (!room.getBuyerEmail().equals(requesterEmail)) {  
    throw new AccessDeniedException("구매자만 거래완료를 신청할 수 있습니다.");  
}  
if (!"거래가능".equals(room.getItemStatus())) {  
    throw new IllegalStateException("이미 처리된 매물입니다.");  
}  
if (room.getCompleteRequestedAt() != null) {  
    throw new IllegalStateException("이미 거래완료를 신청했습니다.");  
}
```

요청한 사람이 이 채팅방의 실제 구매자인지부터 확인합니다. 판매자가 실수(혹은 악의적으로)로 거래완료를 신청하거나, 중복 신청하는 걸 코드 단에서부터 막는 검증 체인입니다.

```
ChatRoom room = Optional.ofNullable(chatRoomMapper.selectOne(roomNo))  
    .orElseThrow(() -> new IllegalArgumentException("존재하지 않는 채팅방입니다: " + roomNo));  
  
if (!room.getSellerEmail().equals(requesterEmail)) {  
    throw new AccessDeniedException("판매자만 거래완료를 확정할 수 있습니다.");  
}  
if (room.getCompleteRequestedAt() == null) {  
    throw new IllegalStateException("구매자가 거래완료를 신청한 이후에만 확정할 수 있습니다.");  
}  
marketItemService.markAsCompleted(room.getItemNo());
```

판매자라도 구매자가 먼저 신청하지 않으면 확정을 못 하도록 막습니다. 한쪽이 일방적으로 거래를 끝낼 수 없게, 반드시 “구매자 신청 -> 판매자 확정” 순서를 강제하는 로직입니다.

시스템 최적화 및 성능 개선 (1)

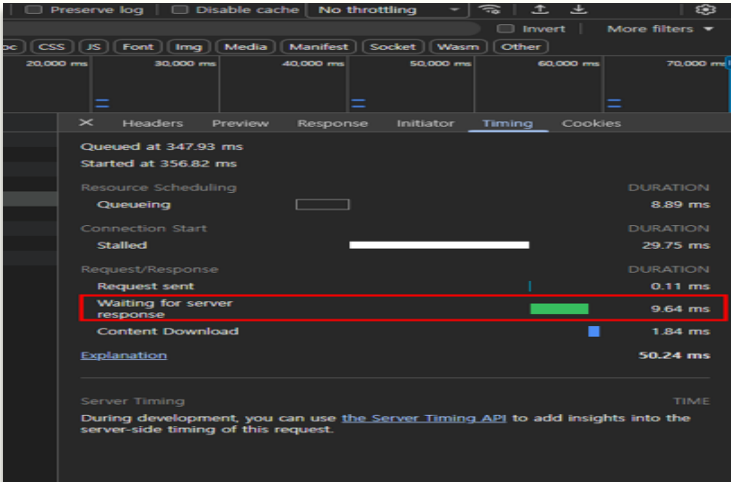
단일 쿼리 통합으로 API 호출과 응답 속도를 획기적으로 개선

Before

Size	Time
0.0 kB	3 ms
0.0 kB	3 ms
0.0 kB	3 ms
0.0 kB	4 ms
0.0 kB	4 ms
0.0 kB	5 ms
0.0 kB	6 ms
0.0 kB	10 ms
0.9 kB	10 ms
0.9 kB	11 ms
0.9 kB	13 ms
0.9 kB	13 ms
0.9 kB	13 ms
0.9 kB	13 ms
0.9 kB	13 ms
0.9 kB	17 ms



After



1. 베이비시터 채팅 – 단일 쿼리 기반 안읽음 बै지

- 쿼리 통합: 채팅방 목록 조회(selectListByMember) 안에 상관 서버 쿼리로 방마다의 안읽음 메시지 수를 함께 계산 - 방 개수와 무관하게 API 호출 1회 유지
- 폴링 최소화: BabysitterLayoutPage에서 20초 주기로 getMyRoomList() 한번만 호출하고, 응답에 담긴 unreadCount를 합산해 배지로 표시
- 설계 원칙: N+1을 나중에 제거한 게 아니라, 채팅 배지 기능을 추가하던 시점부터 애초에 O(1) 단일 쿼리로 설계

지표 (Metric)	방마다 개별 조회했다면 (가정)	실제 구현	효과
API 호출 수 (방 8개 기준)	9회 (목록1+방 별8)	1회	약 89% 감소
응답 시간 합산 (방 8개 기준)	약 141ms	상관 서버 쿼리로 목록 + 안읽음 수 동시 계산	약 14.6배 단축
쿼리 방식	방 별 SELECT 반복	상관 서버 쿼리로 목록 + 안읽음 수 동시 계산	DB 왕복 1회로 통합
폴링 부하	방 개수(N)에 비례	방 개수 무관, 상수	확장성 확보

시스템 최적화 및 성능 개선 (2)

useCallback + lazy()로 재생성·번들 최적화

Before	After
IntersectionObserver 생성: 14266	IntersectionObserver 생성: 1
IntersectionObserver 생성: 14267	
IntersectionObserver 생성: 14268	
IntersectionObserver 생성: 14269	
IntersectionObserver 생성: 14270	
IntersectionObserver 생성: 14271	
IntersectionObserver 생성: 14272	
IntersectionObserver 생성: 14273	
IntersectionObserver 생성: 14274	
IntersectionObserver 생성: 14275	

Before	After
index-CeVuacLK.js	index-C01Xt0_S.js
200	200
script	script
(index):9	(index):9
369 kB	67.5 kB
34 ms	18 ms
	jsx-runtime-D9p77yxo.js
	200
	script
	(index):10
	3.5 kB
	10 ms
	chunk-4ZMWKKQ3-CkF4qWCu.js
	200
	script
	(index):11
	31.6 kB
	11 ms
	redux-toolkit.modern-DYfQWjZw.js
	200
	script
	(index):12
	12.1 kB
	10 ms
	jwtUtil-D1dpxoKx.js
	200
	script
	(index):13
	17.6 kB
	9 ms
	MainOnlyPage-DZ7vJdl.js
	200
	script
	about:client:1
	11.3 kB
	3 ms

2. 리액트 렌더링 최적화

- 무한 스크롤: 스크롤을 맨 아래까지 내리면 자동으로 다음 사진들을 불러와서 이어붙이는 방식
- 코드 스플리팅: 전체 페이지를 한번에 불러오지 않고, 사용자가 실제로 들어간 화면의 코드만 그때그때 불러오는 방식(전체 화면 대부분에 적용)
- useCallback: 스크롤 감지 기능이 화면 다시 그려질 때마다 불필요하게 다시 만들어지지 않도록 고정해서, 스크롤 감지가 끊기거나 오작동하는 문제를 막음

```
// AlbumGridComponent.tsx:44-54
const sentinelRef = useCallback((node: HTMLDivElement | null) => {
  if (observerRef.current) observerRef.current.disconnect();
  observerRef.current = new IntersectionObserver((entries) => {
    if (entries[0].isIntersecting) {
      setPage((prev) => prev + 1);
    }
  });
  if (node) observerRef.current.observe(node);
}, []);
```

```
// recallRouter.tsx (13/14개 라우터에 동일 패턴)
const List = lazy(() => import("../pages/recall/RecallListPage"));
<Suspense fallback={Loading}>...</Suspense>
```

3. AI 산책 추천 - 좌표 기반 캐싱으로 카카오 API 호출 절감

- 캐싱 적용: KakaoLocalClient.searchNearbyParks()에 @Cacheable 적용 — 동일 조건 재요청 시 카카오 API를 다시 호출하지 않음
- 캐시 키 설계: 위·경도를 소수점 3자리로 반올림해 키로 사용 (위치에 따라 약 90~111m 격자) — 같은 동네 사용자끼리 캐시를 공유
- 갱신 주기: Caffeine expireAfterWrite(30분)로 자동 갱신, maximumSize(1000)으로 메모리 상한 설정

```
// KakaoLocalClient.java
@Cacheable(value = "nearbyParks",
    key = "Math.round(#lat*1000)+'_'+Math.round(#lng*1000)
    +'_'+#radiusM")
public List<KakaoPlace> searchNearbyParks(
    double lat, double lng, int radiusM)
```

```
// CacheConfig.java
cacheManager.setCaffeine(
    Caffeine.newBuilder()
        .expireAfterWrite(30, TimeUnit.MINUTES)
        .maximumSize(1000));
```

※ 캐시 설정은 알리지 성분 캐싱과 공유하는 전역 CacheManager를 그대로 사용합니다. □

지표 (Metric)	캐싱 전	캐싱 후
카카오 API 호출	추천 요청마다 매번 1회	동일 격자·30분 이내 재요청 시 0회
캐시 공유 범위	요청자 개별 (공유 없음)	약 100m 격자 내 사용자끼리 공유
캐시 관리	없음	Caffeine, TTL 30분, 최대 1000건



효과
일일 호출 쿼터 절감
캐시 적중률 향상
메모리 사용량 상한 확보

오류해결 (1) – 지자체 지원금대상 정보오염 데이터 분리와 메타데이터 보강, RAG 정확도 개선

1. 이슈 및 문제상황

- 지자체 지원금을 색인할 때 해당 정책의 대상 정보가 아닌 다른 정책의 대상 정보가 저장됨
- 여러 지자체 정책에 동일한 대상 조건이 포함되어 벡터 검색 결과가 왜곡됨
- 실제 지원 대상과 RAG 답변에 표시되는 대상 조건이 불일치하는 문제 발생

2. 원인 분석

- `target` 변수를 중앙부처 지원금 반복문 안에서만 선언함
- 지자체 지원금 반복문에서는 현재 `item`의 대상정보를 가져오지 않고 기존 `target` 변수를 재사용함
- 결과적으로 마지막 중앙부처 지원금의 대상정보가 지자체 지원금 문서와 `Metadata`에 저장됨
- 중앙부처와 지자체 색인로직이 중복되어 필드 초기화 누락이 발생함

3. 해결 방법

- 정책 하나를 변환하는 `_policy_record()` 공통 함수를 생성
- 함수 내부에서 현재 `item`의 대상·생애주기·지역정보를 매번 새로 추출
- 중앙부처와 지자체 데이터를 동일한 변환로직으로 처리
- 지자체 정책이 자신의 대상정보를 사용하는지 검증하는 테스트 추가

4. 결과 및 성과

- 각 지자체 지원금에 해당 정책고유의 대상정보가 정상 저장됨
- 잘못된 대상정보가 임베딩과 `Metadata`에 포함되는 데이터 오염 방지
- 중앙부처·지자체의 중복 색인로직을 공통 함수로 통합해 유지보수성 향상
- 대상·생애주기·지역정보에 대한 자동 테스트를 추가하여 동일 문제의 재발 방지

```
target = str(
    item.get("trgterIndvdlArray")
    or item.get("sprtTrgtCn")
    or ""
).strip()

life_stage = str(
    item.get("lifeArray")
    or item.get("lifeNmArray")
    or ""
).strip()

sido = str(item.get("ctpNm") or "").strip() if local else ""
sigungu = str(item.get("sggNm") or "").strip() if local else ""

document_parts = [f"정책명: {title}"]
if target:
    document_parts.append(f"대상: {target}")
if life_stage:
    document_parts.append(f"생애주기: {life_stage}")

region = " ".join(filter(None, (sido, sigungu)))
if region:
    document_parts.append(f"지역: {region}")

metadata = {
    **_stage_flags(life_stage),
    "target": target,
    "life_stage": life_stage,
    "sido": sido,
    "sigungu": sigungu,
}
```

오류해결 (2) - 문자발송 중복방지 (OpenClaw)

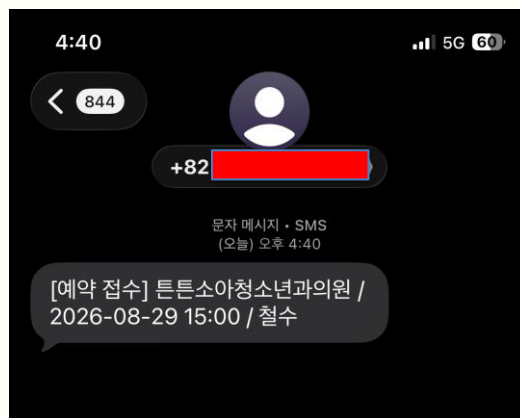
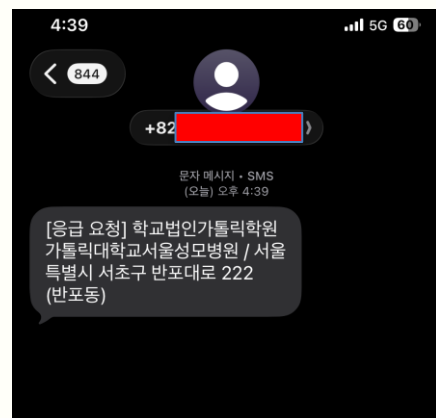
MISSION ID 기반 추적, 중복 방지 구조

1. 이슈 및 문제상황

- LLM 에이전트가 도구를 반복 호출하거나 네트워크 요청이 재시도될 경우 **동일 문자가 여러 번 발송될 가능성** 발생
- 에이전트가 JSON 결과에 설명·Markdown을 추가하면서 Spring에서 결과를 안정적으로 검증 어려움
- 응답 실패로 판단하더라도 Android에서는 이미 문자가 발송됐을 수 있어, **단순 재시도 시 중복 발송 발생 가능**

2. 원인 분석

- 프롬프트로 “한번만 호출” 하도록 시해도 도구 호출 횟수를 완전히 보장할 수 없음
- 최초 구조에서는 동일 요청과 새로운 요청을 구분할 **공통 식별 기준이 부족함**
- Spring, OpenClaw, 네트워크에서 각각 재시도가 발생할 수 있음
- 문자 발송 후 응답이 유실되면 호출자는 발송 성공 여부를 판단하기 어려움
- 중복 방지와 응답 형식을 에이전트 지시에만 의존해 **코드 수준의 강제력이 부족함**



```
// MessageMissionDispatchServiceImpl.java
if (!missionId.equals(resultMissionId)) {
    throw new OpenClawGatewayException(
        "OPENCLAW_MISSION_ID_MISMATCH");
}
```

3. 해결 방법

- Spring에서 요청마다 msg_+UUID 형식의 **고유 missionId** 생성
- 일반 대화와 문자 발송을 분리하고 **message-dispatcher** 전용 워크스페이스 구성
- android_sms_send만 허용하고 미션 JSON 수정 및 다른 도구 호출 금지
- **처리된 missionId와 결과를 캐시하고 동일 ID 요청에는 기존 결과 반환**
- Spring에서 요청 ID와 반환 ID가 다르면 즉시 실패 처리
- 최종 결과는 JSON만 허용하고 파싱할 수 없는 응답은 거부

4. 결과 및 성과

- 같은 missionId가 재전달되면 실제 문자 대신 기존 결과 반환
- 같은 실행에서 도구가 다시 호출되면 즉시 실행 종료
- missionId 변경·다른 미션 결과 반환 시 Spring에서 차단
- **missionId 하나로 Spring → OpenClaw → 플러그인 → Android 전 구간 추적**

오류해결 (3) - AI 육아일기 이미지 리사이징 육아일기 이미지 리사이징 -> 컴퓨터 메모리 최적화

1. 이슈 및 문제상황

- 증상1: 사진을 첨부하여 AI육아일기 자동생성을 돌릴 때 사진에 따라 결과가 나올 때도 있고 응답이 안 올 때도 있는 현상이 발생
- 증상2: 정상적으로 결과가 나오는 경우에도 응답속도의 폭이 커서 사용에 문제가 있다고 판단

2. 원인분석

- 사진 용량 문제: 카메라 사진의 경우 타임아웃이 발생했고 웹에서 캡처 또는 다운로드한 저해상도 사진은 정상 작동하는 걸 발견
- 메모리 사용량 폭주: 컴퓨터 메모리가 100% 가까이 올라감
- 코드 확인: 백엔드 -> 파이썬으로 넘길 때 base64로 변환하여 넘기고 이를 디코딩 후 원본 그대로 VARCO-VISION에게 넘김

```
// DiaryVisionClient.java — 백엔드는 원본 그대로 전송
String base64Image =
Base64.getEncoder().encodeToString(imageBytes);
JsonObject body = new JsonObject();
body.addProperty("imageBase64", base64Image);
HttpRequest request = HttpRequest.newBuilder()
.uri(URI.create(baseUrl + "/api/v1/diary/generate"))
.POST(HttpRequest.BodyPublishers.ofString(body.toString()))
.build();
```

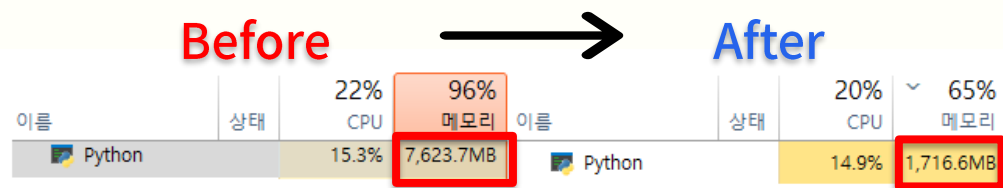
3. 해결방법

- 이미지 리사이징: 파이썬에서 Pillow의 image.thumbnail() 함수를 이용하여 이미지를 1536px로 리사이징하도록 설정하며 이때 1536px보다 작은 경우 원본 그대로 사용할 수 있도록 변경
- 화질 보존: 화질 손실이 가장 적은 방식인 LANCZOS 필터를 사용하여 VARCO-VISION 모델에게 전달할 수 있도록 적용

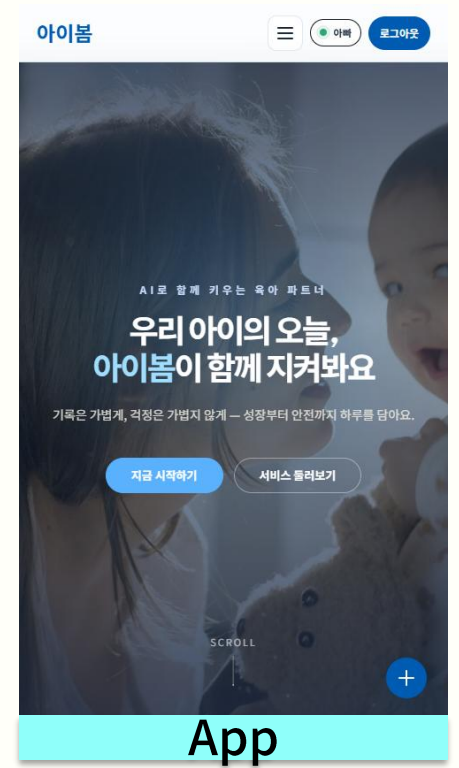
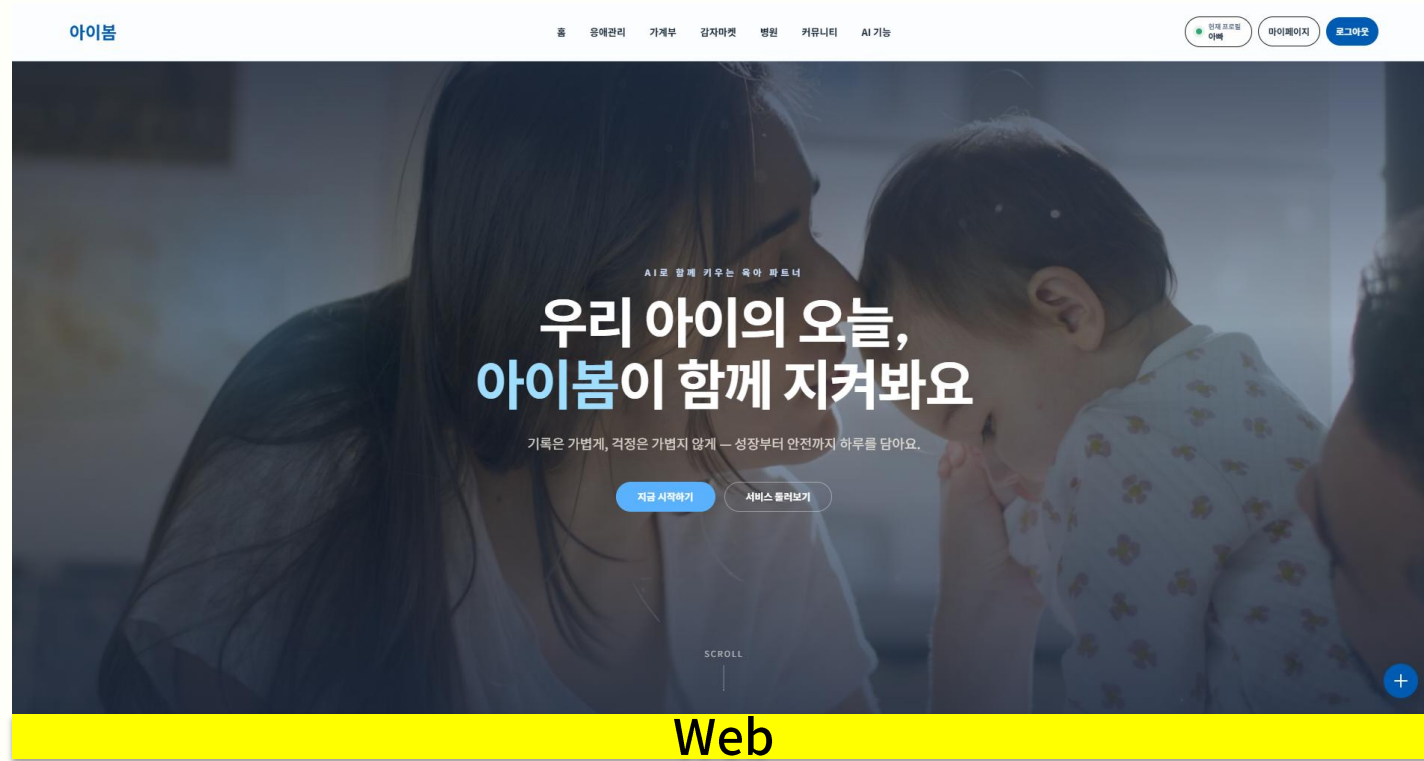
```
# diary_vision_service.py — Python에서 이미지 리사이징
image.thumbnail((MAX_IMAGE_DIMENSION, MAX_IMAGE_DIMENSION),
Image.LANCZOS)
```

4. 결과 및 성과

- 정상작동 확인: 고화질의 사진을 넣어도 타임아웃/메모리 부족 없이 정상작동
- 메모리 안정화: 컴퓨터가 부담을 줄여 최적화 완료
- 처리속도 개선: 이미지 크기가 줄어든 만큼 결과 처리속도가 빨라짐



반응형웹구현



핵심

- PC, 태블릿, 모바일 등 다양한 디바이스 환경에 맞춰 레이아웃이 자동으로 변경되도록 구현하여 어떤 화면에서도 최적의 UI를 제공
- 화면 크기에 따라 컴포넌트의 크기와 배치, 여백을 유동적으로 조정해 가독성과 사용성을 높이고 직관적인 사용자 경험을 제공
- 다양한 해상도와 브라우저 환경에서도 일관된 UI와 UX를 유지할 수 있도록 반응형 웹을 적용하여 접근성과 호환성을 향상

회고

주요 성과

- AI 3종 통합 – Ollama, Google, Vision OCR, Python AI(sentence-transformers, YOLOv8), Spring AI RAG(ChromaDB)까지 하나의 플랫폼에서 연결
- 실시간 커뮤니케이션 – WebSocket(STOMP) 기반 베이비시터, 마켓 실시간 채팅 완성
- 검증된 성능 최적화 – 단일 쿼리 안읽음 배지, 코드 스플리팅(13/14 라우터), 좌표 캐싱, 추론 전 이미지 축소
- Docker Compose 인프라 – 5개 서비스 (MariaDB, Backend, Frontend, Ollama, OpenClaw) 컨테이너 오케스트레이션

아쉬운 점

- 프론트엔드 회귀 테스트 기반 부족 – 백엔드와 AI 서버에는 테스트가 구성되어 있지만, 프론트엔드는 자동화 테스트가 없어 UI 변경에 따른 영향 검증이 수동 테스트에 의존함
향후에는 프론트 기반 에이전트를 활용하여 자동화 테스트를 구현 목표
- 대형 컴포넌트의 책임 집중 – 가계부, 대시보드, AI 추천 화면에서 상태 관리·API 호출·화면 렌더링을 하나의 컴포넌트가 함께 담당해 유지보수 복잡도가 높은 것을 리팩토링을 통하여 안정성 있는 프로젝트를 구현하는 걸로 목표
- 외부 지도 SDK의 타입 안정성 한계 – 카카오맵 객체를 window as any로 여러 기능에서 참조하는 구조를 공통 타입 모듈로 개선하여 안정성을 확보하는 것을 목표

배운 점

- 보안, 설정은 나중에 아니라 처음부터 – JWT_SECRET 같은 필수 환경변수 하나가 빠지면 로그인 전체가 막힌다는 걸 겪으며 설정값 관리 습관의 중요성 체감
- “동작한다” ≠ “안전하다” – 문자 발송 연동에서 같은 요청이 여러 번 재시도 돼도 항상 같은 결과가 나오도록 처음부터 설계해야 함을 체득
- 로컬 전용 산출물은 반드시 자동화 해야 한다 – 벡터DB 색인 데이터처럼 사람이 한번 실행해야 채워지는 데이터는 문서화만으로는 부족하고, 기동 스크립트 차원의 자동 감지, 복구가 필요함을 체감

향후 계획

- 로그인/회원가입 – 카카오 외 소셜 로그인 확장, 휴대폰 본인인증(SMS) 연동으로 이메일 인증만으로는 못 막는 허위가입까지 방지
- 메모이제이션 정책 정립 – 비용이 큰 연산, 리스트 렌더링 위주로 useMemo, React.memo 적용 범위 확대
- AI 클라이언트 통합 – OllamaClient/Spring AI ChatClient 이원 구조를 하나의 추상화로 정리
- 로컬 셋업 자동화 확대 – 벡터DB 자동 색인과 같은 패턴을 다른 로컬 전용 산출물에도 동일하게 적용

THANK

YOU
